SENTINELAI

REMEDIATION & REASONING REPORT

Locally generated evidence document — no paid document-generation API.

Repository	https://github.com/Jayz-yuors/Stock-Insights
Target	Stocks_predictor/app.py
Run ID	20261013-113356-739392
Candidate	sentinelai/security-remediation-20261013-113931
Gate	PASSED

1. Developer Request

changed app.py to include the changes done in calculations.py so that the frontend doesn't face any more errors.

2. Evidence Lineage

MAIN	Original baseline	79015fcf88472223e82bd7347d4d83144479ffdd
LOCAL	Developer proposal	/app/artifacts/ui-uploads/8d303f58b0ff4937a6293e9778532e6d/app.py
AI REVIEW	Existing review	{'summary': 'The proposed code update significantly expands `Stocks_predictor/app.py` by
AI REMEDIATION	Existing remediation	{'finding_id': 'developer-change', 'file_path': 'Stocks_predictor/app.py', 'explanation': 'The de
VERIFICATION	Existing verification	{'verified': True, 'method': 'static', 'reason': 'Target file passed available local syntax verifica
CANDIDATE	Isolated branch	sentinelai/security-remediation-20261013-113931

3. Original vs Local

{

"identical": false,

"developer_reason": "changed app.py to include the changes done in calculations.py so that the frontend doesn't face any more errors.",

"diff": "--- a/Stocks_predictor/app.py\n+++ b/Stocks_predictor/app.py\n@@ -5,21 +5,71 @@\n import plotly.graph_objects as go\n import plotly.express as px\n \n+from plotting import plot_correlation\n from calculations import (\n fetch_prices, fetch_current_price, fetch_company_info,\n compute_sma, compute_ema, detect_abrupt_changes,\n volatility_and_risk, correlation_analysis,\n- compare_companies, get_close_price_column\n+ compare_companies, get_close_price_column,\n+ add_technical_indicators,\n+ prepare_dwm_dataset, summarize_dwm_dataset,\n+ run_dwm_supervised_analysis, build_stock_profiles,\n+ cluster_stock_profiles, build_market_transactions,\n+ mine_association_rules, build_olap_fact_table, olap_aggregate,\n)\n-from plotting import plot_correlation\n-from data_fetcher import get_company_list, run_fetching\n-\n+from data_fetcher import get_company_list, run_fetching, get_latest_date\n+\n+selected_companies = []\n+\n+sector_map = {\n+ \"RELIANCE.NS\": \"Energy & Petrochemicals ■■\",\n+ \"HDFCBANK.NS\": \"Banking & Finance ■\",\n+ \"TCS.NS\": \"IT & Technology ■\",\n+ \"BHARTIARTL.NS\": \"Telecom ■\",\n+ \"ICICIBANK.NS\": \"Banking & Finance ■\",\n+ \"SBIN.NS\": \"Banking & Finance ■\",\n+ \"INFY.NS\": \"IT & Technology ■\",\n+ \"HINDUNILVR.NS\": \"Consumer Goods ■\",\n+ \"LICI.NS\": \"Insurance ■■\",\n+ \"BAJFINANCE.NS\": \"Financial Services ■\",\n+ \"ITC.NS\": \"Diversified FMCG ■\",\n+ \"LT.NS\": \"Infrastructure & Engineering ■■\",\n+ \"MARUTI.NS\": \"Automobiles ■\",\n+ \"HCLTECH.NS\": \"IT & Technology ■\",\n+ \"SUNPHARMA.NS\": \"Pharmaceuticals ■\",\n+ \"KOTAKBANK.NS\": \"Banking & Finance ■\",\n+ \"M&M.NS\": \"Automobiles, Mobility ■\",\n+ \"ULTRACEMCO.NS\": \"Cement & Building Material ■\",\n+ \"AXISBANK.NS\": \"Banking & Finance ■\",\n+ \"NTPC.NS\": \"Power Generation ■\",\n+ \"TITAN.NS\": \"Luxury & Retail ■■\",\n+ \"BAJAJFINSV.NS\": \"Financial Services ■\",\n+ \"HAL.NS\": \"Defence & Aerospace ✈■\",\n+ \"ONGC.NS\": \"Oil & Gas ■\",\n+ \"ADANIPORTS.NS\": \"Logistics & Ports ■\",\n+ \"BEL.NS\": \"Defence Electronics ■■\",\n+ \"WIPRO.NS\": \"IT & Technology ■\",\n+ \"JSWSTEEL.NS\": \"Metals & Steel ■\",\n+ \"TATAMOTORS.NS\": \"Automobiles ■\",\n+ \"ASIANPAINT.NS\": \"Consumer Goods ■\",\n+ \"COALINDIA.NS\": \"Mining & Commodities ■■\",\n+ \"NESTLEIND.NS\": \"Food & Beverages ■\",\n+ \"GRASIM.NS\": \"Materials & Chemicals ■\",\n+ \"HINDALCO.NS\": \"Metals & Mining ■■\",\n+ \"TATASTEEL.NS\": \"Steel Manufacturing ■\",\n+ \"AMBUJACEM.NS\": \"Cement & Construction ■\",\n+ \"RPOWER.NS\": \"Power Generation ■\",\n+ \"DIVISLAB.NS\": \"Pharmaceuticals ■\",\n+ \"SHREECEM.NS\": \"Cement & Building Material ■\",\n+ \"ADANIENT.NS\": \"Diversified ■\"\n+}\n \n # ============== BASIC PAGE CONFIG ==============\n st.set_page_config(\n- page_title=\"■ Stocks Insights\",\n+ page_title=\"Stock Insights – Smart Nifty50 Analytics\",\n+ page_icon=\"■\",\n layout=\"wide\",\n- initial_sidebar_state=\"expanded\"\n+ initial_sidebar_state=\"expanded\",\n)\n \n # ============== THEME TOGGLE ==============\n@@ -34,73 +84,61 @@\n st.session_state[\"theme\"] = theme\n \n if theme == \"Dark\":\n- bg_color = \"#060814\"\n- card_color = \"#101623\"\n- text_color = \"#fafafa\"\n- accent = \"#00b4d8\"\n+ bg_color = \"#060814\"\n+ card_color = \"#101623\"\n+ text_color = \"#fafafa\"\n+ subtext_color = \"rgba(122, 234, 255, 0.85)\"\n+ accent = \"#00e1ff\"\n else:\n- bg_color = \"#f3f8ff\"\n- card_color = \"#ffffff\"\n- text_color = \"#111827\"\n- accent = \"#0077ff\"\n-\n-# ============== GLOBAL CSS + Dev Banner Right ==============\n-st.markdown(\n- f\"\"\"\n+ bg_color = \"#f3f8ff\"\n+ card_color = \"#ffffff\"\n+ text_color = \"#111827\"\n+ subtext_color = \"#444\"\n+ accent = \"#0077ff\"\n+\n+# ============== SEO META TAGS ==============\n+st.markdown(\"\"\"\n+ <meta name=\"description\" content=\"AI-powered stock learning platform — Master Nifty50 analysis with SMA, RSI, volatility, correlation, and smart insights.\">\n+ <meta name=\"keywords\" content=\"Stock Market, Nifty50, Finance Learning, Technical Indicators, SMA, RSI, MACD, Risk & Volatility, AI Stock Analysis\">\n+ <meta property=\"og:title\" content=\"Stock Insights — Master the Market Step-by-Step\">\n+ <meta property=\"og:description\" content=\"Smart Insights for Smarter Investing — Nifty50 analytics powered by AI.\">\n+ <meta property=\"og:type\" content=\"website\">\n+\"\"\", unsafe_allow_html=True)\n+\n+# ============== BRAND HEADER ==============\n+st.markdown(f\"\"\"\n <style>\n- body {{\n- background-color: {bg_color};\n- color: {text_color};\n- }}\n- .main {{\n- background-color: {bg_color};\n- }}\n-\n- /* Developer Banner */\n- .dev-banner {{\n- position: fixed;\n- bottom: 15px;\n- right: -450px;\n- background: linear-gradient(90deg, {accent}, #ff8c42);\n- padding: 10px 22px;\n- font-size: 15px;\n- color: white;\n- border-radius: 999px;\n- font-weight: bold;\n- animation: slideOnce 1.2s ease-out forwards;\n- animation-fill-mode: forwards !important;\n- z-index: 9999;\n- box-shadow: 4px 4px 18px rgba(0,0,0,0.35);\n- }}\n- @keyframes slideOnce {{\n- 0% {{ right: -450px; opacity: 0; }}\n- 100% {{ right: 25px; opacity: 1; }}\n- }}\n+ .brand-title {{\n+ font-size: 42px;\n+ font-weight: 900;\n+ background: linear-gradient(90deg, #12c2e9, #0ee6b7, #00ff95);\n+ -webkit-background-clip: text;\n+ -webkit-text-fill-color: transparent;\n+ text-transform: uppercase;\n+ letter-spacing: 4px;\n+ margin-bottom: 4px;\n+ }}\n+ .brand-tagline {{\n+ font-size: 24px;\n+ font-weight: 700;\n+ color: {text_color};\n+ margin-top: 0px;\n+ margin-bottom: 6px;\n+ }}\n+ .brand-sub {{\n+ font-size: 15px;\n+ font-weight: 500;\n+ color: {subtext_color};\n+ margin-top: 0px;\n+ }}\n </style>\n \n- <div class=\"dev-banner\">\n- Developed by :\n- \n- Jay Keluskar\n- \n- \n- \"\"\",\n- unsafe_allow_html=True,\n-)\n-\n-# ============== HEADER ==============\n-st.markdown(\n- f\"\"\"\n- <h1>■ Stock Insights</h1>\n- \n- Live Nifty50 analytics with MongoDB + Yahoo Finance integration.\n- \n- \"\"\",\n- unsafe_allow_html=True\n-)\n-\n+ <div style=\"text-align:center; margin-top:10px; margin-bottom:30px;\">\n+ <div class=\"brand-title\">Stock Insights \n+ <div class=\"brand-tagline\">Master the Market Step-by-Step \n+ <div class=\"brand-sub\">Nifty50 Analytics — Smart Insights for Smarter Investing \n+ \n+\"\"\", unsafe_allow_html=True)\n \n # ============== AUTO UPDATE ==============\n @st.cache_resource(ttl=24 * 60 * 60)\n@@ -109,7 +147,6 @@\n \n with st.spinner(\"Syncing latest stock data…\"):\n silent_update()\n-\n \n # ============== SIDEBAR FILTERS ==============\n company_list = get_company_list()\n@@ -127,21 +164,152 @@\n st.warning(\"Select at least one company.\")\n st.stop()\n \n-min_date = date(2015, 1, 1)\n-max_date = datetime.today().date()\n-start_date = st.sidebar.date_input(\"Start Date\", min_date)\n-end_date = st.sidebar.date_input(\"End Date\", max_date)\n+MIN_DATE = date(2015, 1, 1)\n+UI_MAX_DATE = date(date.today().year, 12, 31)\n+\n+start_date = st.sidebar.date_input(\n+ \"Start Date\",\n+ value=MIN_DATE,\n+ min_value=MIN_DATE,\n+ max_value="

```
UI_MAX_DATE,\n+)\n+\n+end_date = st.sidebar.date_input(\n+ \"End Date\",\n+ value=date.today(),\n+ min_value=MIN_DATE,\n+ max_value=UI_
MAX_DATE,\n+)\n+\n+if start_date >= end_date:\n+ st.sidebar.error(\"Start date must be earlier than End Date.\")\n+ st.stop()\n+\n+TODAY = date.toda
y()\n+if end_date > TODAY:\n+ end_date = TODAY\n \n date_valid = True\n-if start_date >= end_date:\n- st.sidebar.error(\"Start date must be earlier tha
n end date.\")\n- date_valid = False\n+\n+# ---- Staleness check ----\n+last_updated_dates = []\n+for ticker in selected_companies:\n+ d = get_latest_dat
e(ticker)\n+ if d:\n+ last_updated_dates.append(d)\n+\n+last_updated = (\n+ max(last_updated_dates).strftime(\"%d %b %Y\")\n+ if last_updated_
dates\n+ else \"Unknown\"\n+)\n+\n+stale_msgs = []\n+today = datetime.now().date()\n+for t in selected_companies:\n+ d = get_latest_date(t)\n+ if d:\
n+ delay = (today - d).days\n+ if delay >= 2:\n+ icon = \"■\" if delay > 5 else \"■■\"\n+ stale_msgs.append(f\"{icon} <strong>{t}</stron
g> — {delay} days ago\")\n+\n+if stale_msgs:\n+ msg_html = \"<br>\".join(stale_msgs)\n+ st.sidebar.markdown(f\"\"\"\n+ <div style=\"\n+ backgrou
nd:rgba(255,180,0,0.12);\n+ border-left:4px solid #ffae00;\n+ padding:12px 14px;\n+ border-radius:8px;\n+ margin-top:20px;\n+ color:
#ffe7c4;\">\n+ <strong>■ Update Delay Detected</strong><br>\n+ <span style='font-size:13px;'>{msg_html}</span>\n+ </div>\n+ \"\"\", unsafe
_allow_html=True)\n+else:\n+ st.sidebar.markdown(f\"\"\"\n+ <div style=\"\n+ background:rgba(0,255,140,0.10);\n+ border-left:4px solid #00ff9d;
\n+ padding:12px 14px;\n+ border-radius:8px;\n+ margin-top:20px;\n+ color:#baffdd;\">\n+ ■ All selected stocks are updated till the lat
est available date!\n+ </div>\n+ \"\"\", unsafe_allow_html=True)\n+\n+# ============== FOOTER ==============\n+\n+# ============== FOO
TER (Updated daily, no developer name) ==============\n+st.markdown(\n+ f\"\"\"\n+ <div style=\"\n+ text-align:center;\n+ padding:10px;\n+
margin-top:30px;\n+ font-size:14px;\n+ font-weight:500;\n+ color:#a6fff5;\n+ border-top:1px solid rgba(255,255,255,0.15);\n+ \">\n+
<span style='font-size:13px; color:#9cdcff;'>\n+ ■ Updated daily at <strong>10:00 AM</strong> & <strong>11:00 PM IST</strong><br>\n+
■ Data available till: <strong style='color:#7afcff;'>{last_updated}</strong>\n+ </span>\n+ </div>\n+ \"\"\",\n+ unsafe_allow_html=True,\n+)\n+\n+\
n+if selected_companies:\n+ st.sidebar.markdown(\"\"\"\n+ <div style=\"\n+ background:rgba(0,255,200,0.06);\n+ border-left:4px solid #00ffc8;\n
+ padding:10px 14px;\n+ border-radius:6px;\n+ margin-top:14px;\n+ color:#b9fff4;\n+ \">\n+ <strong>■■ Selected Stocks:</strong>\n+
</div>\n+ \"\"\", unsafe_allow_html=True)\n+\n+ for t in selected_companies:\n+ sector = sector_map.get(t, \"Sector Not Available ■■\")\n+ st.
sidebar.markdown(\n+ f\"<span style='color:#d9f7ff;'>• <strong>{t}</strong> — {sector}</span>\",\n+ unsafe_allow_html=True,\n+ )\n+\n+s
t.sidebar.markdown(\"\"\"\n+<div style=\"\n+ background:rgba(0,255,200,0.06);\n+ border-left:4px solid #00ffc8;\n+ padding:10px 14px;\n+ border-rad
ius:6px;\n+ margin-top:14px;\n+ color:#b9fff4;\n+\">\n+<strong>■ Date Range:</strong><br>\n+\"\"\", unsafe_allow_html=True)\n+\n+st.sidebar.markdo
wn(\n+ f\"<span style='color:#d9f7ff;'>• From: <strong>{start_date}</strong></span><br>\"\n+ f\"<span style='color:#d9f7ff;'>• To: <strong>{end_date}</
strong></span>\",\n+ unsafe_allow_html=True,\n+)\n+st.sidebar.markdown(\"</div>\", unsafe_allow_html=True)\n \n \n # ============== Helper Functi
ons ==============\n def build_budget_options():\n- small = list(range(0, 100001, 10000))\n- large = list(range(200000, 1000001, 100000))\n+ smal
l = list(range(0, 100001, 10000))\n+ large = list(range(200000, 1000001, 100000))\n values = small + large\n labels = [f\"■{v:,.0f}\" for v in values]\n
return values, labels\n@@ -152,15 +320,15 @@\n return 50, \"Hold\", 0, 0\n \n lookback = 60 if horizon == \"Short Term\" else 180\n- recent = df
.tail(min(len(df), lookback))\n-\n- x = np.arange(len(recent))\n- y = recent[col_close].values\n- slope, _ = np.polyfit(x, y, 1)\n+ recent = df.tail(min(len(
df), lookback))\n+\n+ x = np.arange(len(recent))\n+ y = recent[col_close].values\n+ slope, _ = np.polyfit(x, y, 1)\n pct_change = (y[-1] - y
[0]) / y[0] * 100\n- vol = recent[col_close].pct_change().std() * 100\n-\n- conf = 50 + pct_change/2 - vol/4\n+ vol = recent[col_close].pct_change().st
d() * 100\n+\n+ conf = 50 + pct_change / 2 - vol / 4\n conf = float(max(5, min(95, conf)))\n \n if pct_change > 5 and slope > 0:\n@@ -184,82 +352,287
@@\n if len(recent) < 10:\n return pd.DataFrame(), pd.DataFrame()\n \n- x = np.arange(len(recent))\n- y = recent[col_close].values\n+ x
= np.arange(len(recent))\n+ y = recent[col_close].values\n slope, intercept = np.polyfit(x, y, 1)\n \n- future_days = 15 if horizon == \"Short
Term\" else 60\n- last_date = df[\"trade_date\"].iloc[-1]\n- future_dates = pd.bdate_range(last_date + pd.Timedelta(days=1), periods=future_days)\n- x_
future = np.arange(len(recent), len(recent) + len(future_dates))\n+ future_days = 15 if horizon == \"Short Term\" else 60\n+ last_date = df[\"trade_dat
e\"].iloc[-1]\n+ future_dates = pd.bdate_range(last_date + pd.Timedelta(days=1), periods=future_days)\n+ x_future = np.arange(len(recent), len(rece
nt) + len(future_dates))\n future_prices = intercept + slope * x_future\n \n- future_df = pd.DataFrame({\"trade_date\": future_dates, col_close: future_pric
es})\n- future_df[\"SMA\"] = future_df[col_close].rolling(20, min_periods=1).mean()\n-\n- buy = future_df[future_df[col_close] < future_df[\"SMA\"]]\n+ fut
ure_df = pd.DataFrame({\"trade_date\": future_dates, col_close: future_prices})\n+ future_df[\"SMA\"] = future_df[col_close].rolling(20, min_periods
=1).mean()\n+\n+ buy = future_df[future_df[col_close] < future_df[\"SMA\"]]\n sell = future_df[future_df[col_close] > future_df[\"SMA\"]]\n return buy, s
ell\n \n \n+def download_csv(df: pd.DataFrame, name: str):\n+ \"\"\"Reusable download helper for tabular data.\"\"\"\n+ st.download_button(\n+ label
=\"■ Download this data as CSV\",\n+ data=df.to_csv(index=False),\n+ file_name=f\"{name}.csv\",\n+ mime=\"text/csv\",\n+ )\n+\n+\n # ====
========== TABS ==============\n-tab1, tab2, tab3, tab4, tab5 = st.tabs([\n+tab1, tab2, tab3, tab4, tab5, tab6, tab7, tab8, tab9, tab10 = st.tabs([\n \"
■ Price Trends\",\n \"■ Abrupt Changes\",\n \"■ Risk & Volatility\",\n \"■ Compare & Correlate\",\n- \"■ Smart Insights\"\n+ \"■ Smart Insights\",\n
+ \"■ DWM: Explore & Clean\",\n+ \"■ DWM: Classification & Regression\",\n+ \"■ DWM: Clustering\",\n+ \"■ DWM: Apriori Rules\",\n+ \"■ DWM:
OLAP Cube\",\n ])\n \n \n-# ============== TAB 1 — Price Trends ==============\n+# ============== TAB 1 — Price Trends + Indicators ======
========\n with tab1:\n- st.subheader(\"Price Trend + SMA & EMA\")\n-\n- if date_valid:\n- for ticker in selected_companies:\n- df = fetch_pr
ices(ticker, start_date, end_date)\n-\n- if df is None or df.empty:\n- st.warning(f\"No data for {ticker}\")\n- continue\n-\n- df =
compute_sma(df)\n- df = compute_ema(df)\n- col_close = get_close_price_column(df)\n-\n- st.markdown(f\"### {ticker}\")\n- st.m
etric(\"Latest Price\", f\"■ {df[col_close].iloc[-1]:.2f}\")\n- st.line_chart(df.set_index(\"trade_date\")[[col_close, \"SMA\", \"EMA\"]])\n-\n-\n-# ==========
==== TAB 2 — Improved Abrupt Changes ==============\n+ st.subheader(\"■ Price Trend + Technical Indicators\")\n+\n+ st.markdown(\"### ■ Lear
n & Analyze with Price Trends\")\n+\n+ col1, col2 = st.columns(2)\n+ with col1:\n+ st.markdown(\"\"\"\n+ <div style=\"\n+ background:rgba
(0,180,216,0.08);\n+ border-left:4px solid #00b4d8;\n+ padding:12px 16px;\n+ border-radius:8px;\n+ color:#c9e8ff;\n+ mar
gin-bottom:12px;\n+ \">\n+ <strong>■ What will you analyze here?</strong><br>\n+ • Stock price behaviour & market cycles<br>\n+ • Spo
t big uptrends & downtrends<br>\n+ • Understand momentum using SMA/EMA<br>\n+ • Validate trend direction confidently\n+ </div>\n+ \"
\"\", unsafe_allow_html=True)\n+\n+ with col2:\n+ st.markdown(\"\"\"\n+ <div style=\"\n+ background:rgba(0,180,216,0.08);\n+ borde
r-left:4px solid #00b4d8;\n+ padding:12px 16px;\n+ border-radius:8px;\n+ color:#c9e8ff;\n+ margin-bottom:12px;\n+ \">\n+
<strong>■ RSI — Momentum Pressure</strong><br>\n+ • RSI > 70 = Overbought → Possible correction<br>\n+ • RSI < 30 = Oversold → Possi
ble bounce<br>\n+ • Helps time better entry & exit points\n+ </div>\n+ \"\"\", unsafe_allow_html=True)\n+\n+ col3, col4 = st.columns(2)\n+
with col3:\n+ st.markdown(\"\"\"\n+ <div style=\"\n+ background:rgba(0,180,216,0.08);\n+ border-left:4px solid #00b4d8;\n+ pad
ding:12px 16px;\n+ border-radius:8px;\n+ color:#c9e8ff;\n+ margin-bottom:12px;\n+ \">\n+ <strong>■ SMA / EMA — Trend Dir
ection</strong><br>\n+ • Price above MA → Bullish momentum<br>\n+ • Price below MA → Bearish weakness<br>\n+ • Short SMAs react fast
er to volatility\n+ </div>\n+ \"\"\", unsafe_allow_html=True)\n+\n+ with col4:\n+ st.markdown(\"\"\"\n+ <div style=\"\n+ background:r
gba(0,180,216,0.08);\n+ border-left:4px solid #00b4d8;\n+ padding:12px 16px;\n+ border-radius:8px;\n+ color:#c9e8ff;\n+
margin-bottom:12px;\n+ \">\n+ <strong>■ MACD — Trend Confidence</strong><br>\n+ • Upward crossover → Strong buying zone<br>\n+
• Downward crossover → Weakening trend<br>\n+ • Great for breakout confirmation\n+ </div>\n+ \"\"\", unsafe_allow_html=True)\n+\n+ st.
markdown(\"\"\"\n+ <div style=\"\n+ background:rgba(0,180,216,0.05);\n+ border-left:4px solid #00b4d8;\n+ padding:14px 18px;\n+ borde
r-radius:8px;\n+ margin-top:6px;\n+ color:#aee6ff;\n+ \">\n+ <strong>■ Why this matters?</strong><br>\n+ You learn trend-following strategie
s and understand <strong>market psychology</strong> —\n+ helping you decide <strong>when to enter</strong> and <strong>when to exit</strong> sma
rtly.\n+ </div>\n+ \"\"\", unsafe_allow_html=True)\n+\n+ view_mode = st.radio(\n+ \"Indicator Display Mode\",\n+ [\"Overlay on Chart\", \"Separ
ate Panels\"],\n+ horizontal=True,\n+ )\n+\n+ ma_options = {\n+ \"20 SMA\": \"SMA_20\",\n+ \"50 SMA\": \"SMA_50\",\n+ \"200 SMA\":
\"SMA_200\",\n+ \"20 EMA\": \"EMA_20\",\n+ \"50 EMA\": \"EMA_50\",\n+ }\n+\n+ overlay_selected = st.multiselect(\n+ \"Select Moving Av
erages\",\n+ list(ma_options.keys()),\n+ default=[\"20 SMA\", \"50 SMA\"],\n+ )\n+\n+ osc_options = [\"RSI (14)\", \"MACD\"]\n+ osc_selected
= st.multiselect(\n+ \"Oscillators (separate charts)\",\n+ osc_options,\n+ default=[\"RSI (14)\"],\n+ )\n+\n+ for ticker in selected_companies:\n
+ df = fetch_prices(ticker, start_date, end_date)\n+ if df is None or df.empty:\n+ continue\n+\n+ df = add_technical_indicators(df)\n+
col_close = get_close_price_column(df)\n+ last_valid_close = df[col_close].dropna().iloc[-1] if df[col_close].notna().any() else None\n+\n+ st.ma
rkdown(f\"### ■ {ticker}\")\n+ st.metric(\n+ \"Latest Close Price\",\n+ f\"■ {last_valid_close:.2f}\" if last_valid_close is not None else \"Unav
ailable\",\n+ )\n+\n+ if view_mode == \"Overlay on Chart\":\n+ fig = go.Figure()\n+ fig.add_trace(go.Scatter(x=df.trade_date, y=df[col_c
```

```
lose],\n+                name=\"Close Price\", mode=\"lines\"))\n+            for label in overlay_selected:\n+                col = ma_options[label]\n+                if col in df.columns:\n+                    fig.add_trace(go.Scatter(x=df.trade_date, y=df[col],\n+                                             name=label, mode=\"lines\"))\n+            fig.update_layout(\n+                title=f\"{ticker} — Price + Indicators\",\n+                height=500,\n+                margin=dict(t=80, b=10),\n+                legend=dict(orientation=\"h\", yanchor=\"bottom\", y=1.10,\n+                            xanchor=\"center\", x=0.5),\n+                xaxis_title=\"Date\",\n+                yaxis_title=\"Price (■)\",\n+            )\n+            st.plotly_chart(fig, use_container_width=True)\n+            download_csv(df, f\"{ticker}_overlay_data\")\n+\n+        else:\n+            st.markdown(\"#### ■ Close Price\")\n+            st.line_chart(df.set_index(\"trade_date\")[col_close], height=350)\n+            for label in overlay_selected:\n+                col = ma_options[label]\n+                if col in df.columns:\n+                    st.markdown(f\"#### ■ {label}\")\n+                    st.line_chart(df.set_index(\"trade_date\")[[col_close, col]], height=300)\n+\n+        if \"RSI (14)\" in osc_selected:\n+            st.markdown(\"#### ■ RSI (14)\")\n+            st.line_chart(df.set_index(\"trade_date\")[\"RSI_14\"], height=250)\n+\n+        if \"MACD\" in osc_selected:\n+            st.markdown(\"#### ■ MACD Indicator\")\n+            st.line_chart(df.set_index(\"trade_date\")[\"MACD\"], height=250)\n+\n+        with st.expander(\"■ View & Download Data\"):\n+            st.dataframe(df, use_container_width=True)\n+            download_csv(df, f\"{ticker}_indicators\")\n+\n+\n+# ============== TAB 2 — Abrupt Changes ==============\n with tab2:\n    st.subheader(\"Sudden Price Jumps & Falls ■\")\n+    st.markdown(f\"\"\"\n+    <div style=\"\n+        background:rgba(255,136,0,0.08);\n+        border:1px solid rgba(255,180,50,0.4);\n+        border-radius:10px;\n+        padding:12px 18px;\n+        text-align:right;\n+        font-size:14.5px;\n+        line-height:1.45;\n+        color:{text_color};\n+        margin:0px 0px 14px 0px;\n+    \">\n+        ■ Sudden price swings occur due to <strong>breaking news</strong>,\n+        <strong>big-money trades</strong>, or <strong>volume spikes</strong>.<br>\n+        ■ Upward jumps with strength → potential <strong>early breakout</strong>.<br>\n+        ■ Sharp drops → fear selling & <strong>trend reversal</strong> risk.\n+    </div>\n+\n+    <div style=\"\n+        background:rgba(255,136,0,0.07);\n+        border-left:4px solid #ff8800;\n+        border-radius:8px;\n+        padding:10px 16px;\n+        text-align:left;\n+        font-size:14.5px;\n+        line-height:1.45;\n+        color:{text_color};\n+    \">\n+        <strong>■ Why this matters?</strong><br>\n+        Spot <strong>breakout chances</strong> early or protect yourself before\n+        <strong>fast crashes</strong> cause damage ■■.\n+    </div>\n+    \"\"\", unsafe_allow_html=True)\n \n    threshold_pct = st.slider(\"Threshold (%)\", 3, 20, 7) / 100\n \n    for ticker in selected_companies:\n        df = fetch_prices(ticker, start_date, end_date)\n-        if df is None or df.empty: continue\n+        if df is None or df.empty:\n+            continue\n        col_close = get_close_price_column(df)\n-\n-        abrupt = detect_abrupt_changes(df, threshold_pct)\n-        st.markdown(f\"### {ticker}\")\n-\n+        abrupt = detect_abrupt_changes(df, threshold_pct)\n+\n+        st.markdown(f\"### {ticker} — Abrupt Moves\")\n        if abrupt.empty:\n            st.info(\"No major movements detected.\")\n            continue\n \n-        # ■ Highlight chart\n        fig = px.bar(\n            abrupt,\n            x=\"trade_date\",\n            y=\"pct_change\",\n            color=\"pct_change\",\n            color_continuous_scale=\"RdYlGn\",\n-            title=f\"Abrupt % Movements — {ticker}\"\n+            title=f\"Abrupt % Movements — {ticker}\",\n        )\n+        fig.update_layout(xaxis_title=\"Date\", yaxis_title=\"Daily % Change\",\n+                          coloraxis_colorbar_title=\"% Change\")\n        st.plotly_chart(fig, use_container_width=True)\n        st.dataframe(abrupt, use_container_width=True)\n \n@@ -267,24 +640,127 @@\n # ============== TAB 3 — Risk View ==============\n with tab3:\n    st.subheader(\"Volatility & Risk Index\")\n+    st.markdown(\"\"\"\n+    <div style=\"\n+        background:linear-gradient(135deg, rgba(255,40,40,0.10), rgba(180,0,0,0.08));\n+        padding:16px 18px;\n+        border-radius:10px;\n+        border-left:4px solid #ff4d4d;\n+        margin-bottom:10px;\n+        color:#ffdddd;\n+    \">\n+    <strong>■ Volatility means uncertainty:</strong><br>\n+    • Big price swings → confusion / hype ■■■<br>\n+    • High risk during panic buying/selling ■<br>\n+    • Sudden drops → instability warning ■<br>\n+    </div>\n+\n+    <div style=\"\n+        background:rgba(255,255,255,0.04);\n+        padding:14px 18px;\n+        border-radius:10px;\n+        border-left:4px solid #ff9f43;\n+        margin-bottom:10px;\n+        color:#ffe9c7;\n+    \">\n+    <strong>■ How we measure Risk:</strong><br>\n+    • Volatility Index (fear indicator) ■<br>\n+    • Price variance → instability levels ■<br>\n+    • Shorter time periods → more sensitivity ■■<br>\n+    </div>\n+\n+    <div style=\"\n+        background:rgba(255,255,255,0.03);\n+        padding:14px 18px;\n+        border-radius:10px;\n+        border-left:4px solid #ffd166;\n+        color:#fff7d6;\n+    \">\n+    <strong>■ Why this matters?</strong><br>\n+    Volatility helps you judge <strong>when to reduce position size</strong> and\n+    <strong>protect capital</strong> when the market heats up ■■.\n+    </div>\n+    \"\"\", unsafe_allow_html=True)\n \n    window = st.slider(\"Volatility Window\", 5, 55, 20)\n-\n    for ticker in selected_companies:\n        df = fetch_prices(ticker)\n+        if df is None or df.empty:\n+            continue\n        vr = volatility_and_risk(df, window)\n-        st.markdown(f\"### {ticker}\")\n+        st.markdown(f\"### {ticker}\")\n+        st.line_chart(vr.set_index(\"trade_date\")[[\"volatility\", \"risk\"]])\n+        st.markdown(f\"### {ticker} — Volatility vs Risk\")\n        st.line_chart(vr.set_index(\"trade_date\")[[\"volatility\", \"risk\"]],\n                      use_container_width=True)\n \n \n# ============== TAB 4 — Comparison View ==============\n with tab4:\n    st.subheader(\"Compare Price Trends & Correlation\")\n-\n    if len(selected_companies) >= 2 and date_valid:\n+        st.markdown(\"\"\"\n+        <div style=\"margin-top:10px;\">\n+        <div style=\"\n+            background:linear-gradient(145deg, #005bea33, #00c6fb22);\n+            padding:16px 20px;\n+            border-radius:10px;\n+            color:#e8f8ff;\n+            margin-bottom:12px;\n+            border-left:4px solid #00c6fb;\n+        \">\n+        <strong>■ Compare Price Strength</strong><br>\n+        • Normalize price → fair comp. ■<br>\n+        • Spot consistent outperformers ■<br>\n+        • Check who beats the market over time ■■<br>\n+        </div>\n+\n+        <div style=\"\n+            background:linear-gradient(145deg, #b721ff33, #21d4fd22);\n+            padding:16px 20px;\n+            border-radius:10px;\n+            color:#f1e6ff;\n+            margin-bottom:14px;\n+            border-left:4px solid #9b59ff;\n+        \">\n+        <strong>■ Correlation Outlook</strong><br>\n+        • Same direction = paired performance ■<br>\n+        • Opposite direction = hedge opportunities ■<br>\n+        • Build diversified portfolios with confidence ■■<br>\n+        </div>\n+\n+        <div style=\"\n+            background:rgba(0,200,255,0.06);\n+            padding:12px 16px;\n+            border-radius:10px;\n+            border-left:4px solid #00c6fb;\n+            color:#c7f4ff;\n+        \">\n+        <strong>■ Why this matters?</strong><br>\n+        Avoid <strong>putting all eggs in one basket</strong> → diversify smartly to reduce risk ■■■.\n+        </div>\n+        </div>\n+        \"\"\", unsafe_allow_html=True)\n+\n+\n        if len(selected_companies) >= 2:\n            merged = compare_companies(selected_companies, start_date, end_date)\n-            st.line_chart(merged)\n-\n+            st.markdown(\"### ■ Normalized Price Comparison\")\n+\n+            fig = go.Figure()\n+            for ticker in selected_companies:\n+                if ticker in merged.columns:\n+                    fig.add_trace(go.Scatter(x=merged.index, y=merged[ticker],\n+                                             mode=\"lines\", name=ticker))\n+            fig.update_layout(\n+                xaxis_title=\"Date\",\n+                yaxis_title=\"Normalized Price\",\n+                title=\"Price Comparison Across Selected Stocks\",\n+                height=550,\n+                margin=dict(l=0, r=0, t=40, b=80),\n+                legend=dict(orientation=\"h\", yanchor=\"top\", y=-0.15,\n+                            xanchor=\"center\", x=0.5, title=None),\n+            )\n+            st.plotly_chart(fig, use_container_width=True)\n+            download_csv(merged.reset_index(), \"normalized_price_comparison\")\n+\n+\n+            st.markdown(\"### ■ Correlation Matrix\")\n            corr = correlation_analysis(selected_companies)\n            st.dataframe(corr.style.background_gradient(cmap=\"coolwarm\"))\n            plot_correlation(corr)\n@@ -292,61 +768,357 @@\n \n # ============== TAB 5 — Smart Insights ==============\n with tab5:\n-    st.subheader(\"Smart Insights, Opportunities & Forecast\")\n+    st.subheader(\"■ Smart Insights, Opportunities & Forecast\")\n+    st.markdown(\"### ■ Understand AI-Driven Market Signals\")\n+    col1, col2 = st.columns(2)\n+    with col1:\n+        st.markdown(\"\"\"\n+        <div style=\"\n+            background:rgba(0,255,200,0.08);\n+            border-left:4px solid #00ffc8;\n+            padding:12px 16px;\n+            border-radius:8px;\n+            color:#b9fff4;\n+            margin-bottom:12px;\n+        \">\n+        <strong>■ AI Trend Confidence</strong><br>\n+        • Tags a stock as Buy / Hold / Risky ■<br>\n+        • Reduces emotional trading mistakes\n+        </div>\n+        \"\"\", unsafe_allow_html=True)\n+\n+    with col2:\n+        st.markdown(\"\"\"\n+        <div style=\"\n+            background:rgba(0,255,200,0.08);\n+            border-left:4px solid #00ffc8;\n+            padding:12px 16px;\n+            border-radius:8px;\n+            color:#b9fff4;\n+            margin-bottom:12px;\n+        \">\n+        <strong>■ Future Price Tendency</strong><br>\n+        • Shows short-term directional pressure<br>\n+        • Improves timing for smarter entries\n+        </div>\n+        \"\"\", unsafe_allow_html=True)\n+\n+    col3, col4 = st.columns(2)\n+    with col3:\n+        st.markdown(\"\"\"\n+        <div style=\"\n+            background:rgba(0,255,200,0.08);\n+            border-left:4px solid #00ffc8;\n+            padding:12px 16px;\n+            border-radius:8px;\n+            color:#b9fff4;\n+            margin-bottom:12px;\n+        \">\n+        <strong>■ Buy & Sell Zones</strong><br>\n+        • Shows where reversals are likely<br>\n+        • Avoid buying tops and selling bottoms\n+        </div>\n+        \"\"\", unsafe_allow_html=True)\n+\n+    with col4:\n+        st.markdown(\"\"\"\n+        <div style=\"\n+            background:rgba(0,255,200,0.08);\n+            border-left:4px solid #00ffc8;\n+            padding:12px 16px;\n+            border-radius:8px;\n+            color:#b9fff4;\n+            margin-bottom:12px;\n+        \">\n+        <strong>■ Share Affordability</strong><br>\n+        • Teaches sizing and discipline<br>\n+        • Plan smarter investments\n+        </div>\n+        \"\"\", unsafe_allow_html=True)\n+\n+    st.markdown(\"\"\"\n+    <div style=\"\n+        background:rgba(0,255,200,0.05);\n+        border-left:4px solid #00ffc8;\n+        padding:14px 18px;\n+        border-radius:8px;\n+        margin-top:6px;\n+        color:#a6fff5;\n+    \">\n+    <strong>■ Why this matters?</strong><br>\n+    Builds <strong>high-confidence decisions</strong> and prevents\n+    <strong>FOMO or panic selling</strong> — clarity on\n+    <strong>when to enter</strong> & <strong>exit</strong> the market like a smart investor.\n+    </div>\n+    \"\"\", unsafe_allow_html=True)\n+\n+    st.caption(\"Educational purpose only — Not for financial decisions ■\")\n \n+    budget_vals, budget_labels = build_budget_options()\n+    col_a, col_b, col_c = st.columns(3)\n+    with col_a:\n+        budget_label = st.selectbox(\"Budget\", budget_labels, index=4)\n+    budget = budget_vals[budget_labels.index(budget_label)]\n+    with col_b:\n+        horizon = st.radio(\"Type\", [\"Short Term\", \"Long Term\"], horizontal=True)\n+    with col_c:\n+        forecast_window = 15 if horizon == \"Short Term\" else 60\n+        st.metric(\"Forecast\", f\"{forecast_window} days\")\n+\n+    st.caption(\
```

```
"Based on DB trends — Not financial advice.Do not invest solely on this.Just for educational purpose !\")\n+    budget      = budget_vals[4]\n+    budget_label = budget_labels[4]\n+    horizon     = \"Short Term\"\n+\n+    colA, colB, colC = st.columns(3)\n+    with colA:\n+        budget_label = st.selectbox(\"Budget\", budget_labels, index=4)\n+        budget       = budget_vals[budget_labels.index(budget_label)]\n+    with colB:\n+        horizon = st.radio(\"Type\", [\"Short Term\", \"Long Term\"], horizontal=True)\n+    with colC:\n+        forecast_window  = 15 if horizon == \"Short Term\" else 60\n+        st.metric(\"Forecast Window\", f\"{forecast_window} days\")\n+\n+    st.caption(\"Based on trends — Not financial advice. Educational use only.\")\n \n    for ticker in selected_companies:\n        df = fetch_prices(ticker)\n-        df = compute_sma(df)\n+        if df is None or df.empty:\n+            continue\n+\n+        df       = compute_sma(df)\n        col_close = get_close_price_column(df)\n \n        conf, label, pct, vol = analyze_trend_confidence(df, col_close, horizon)\n-        latest = df[col_close].iloc[-1]\n-        shares = int(budget / latest)\n+        valid_prices = (\n+            pd.to_numeric(df[col_close], errors=\"coerce\")\n+            .replace([np.inf, -np.inf], np.nan)\n+            .dropna()\n+        )\n+        if valid_prices.empty or valid_prices.iloc[-1] <= 0:\n+            st.warning(f\"{ticker}: no valid latest closing price is available.\")\n+            continue\n+\n+        latest = float(valid_prices.iloc[-1])\n+        shares = max(1, int(budget / latest))\n \n        st.markdown(f\"---\\n### {ticker}\")\n-        m1, m2, m3, m4 = st.columns(4)\n-        m1.metric(\"Signal\", label)\n-        m2.metric(\"Confidence\", f\"{conf:.1f}%\")\n-        m3.metric(\"Trend %\", f\"{pct:+.2f}%\")\n-        m4.metric(\"Volatility\", f\"{vol:.2f}%\")\n-\n-        st.caption(f\"With {budget_label}, Approx shares: **{shares}**\")\n+        col1, col2, col3, col4 = st.columns(4)\n+        col1.metric(\"Signal\", label)\n+        col2.metric(\"Confidence\", f\"{conf:.1f}%\")\n+        col3.metric(\"Trend %\", f\"{pct:+.2f}%\")\n+        col4.metric(\"Volatility\", f\"{vol:.2f}%\")\n+\n+        st.caption(f\"With {budget_label}, Approx shares possible: **{shares}**\")\n \n \n        buy_future, sell_future = project_future(df, col_close, horizon)\n+\n        fig = go.Figure()\n-        fig.add_trace(go.Scatter(x=df[\"trade_date\"], y=df[col_close], mode=\"lines\", name=\"Close\"))\n-        fig.add_trace(go.Scatter(x=df[\"trade_date\"], y=df[\"SMA\"], mode=\"lines\", name=\"SMA\"))\n-        fig.add_trace(go.Scatter(x=buy_future[\"trade_date\"], y=buy_future[col_close],\n-                                 mode=\"markers\", marker_color=\"green\", name=\"Future Buy\"))\n-        fig.add_trace(go.Scatter(x=sell_future[\"trade_date\"], y=sell_future[col_close],\n-                                 mode=\"markers\", marker_color=\"red\", name=\"Future Sell\"))\n-\n-        fig.update_layout(title=f\"{ticker} — Forecasted Zones\")\n+        fig.add_trace(go.Scatter(x=df[\"trade_date\"], y=df[col_close],\n+                                 mode=\"lines\", name=\"Close Price\"))\n+        fig.add_trace(go.Scatter(x=df[\"trade_date\"], y=df[\"SMA\"],\n+                                 mode=\"lines\", name=\"SMA\"))\n+\n+\n+        if buy_future is not None and not buy_future.empty:\n+            fig.add_trace(go.Scatter(x=buy_future[\"trade_date\"],\n+                                     y=buy_future[col_close],\n+                                     mode=\"markers\", name=\"Future Buy\",\n+                                     marker_color=\"green\"))\n+        if sell_future is not None and not sell_future.empty:\n+            fig.add_trace(go.Scatter(x=sell_future[\"trade_date\"],\n+                                     y=sell_future[col_close],\n+                                     mode=\"markers\", name=\"Future Sell\",\n+                                     marker_color=\"red\"))\n+\n+        fig.update_layout(title=f\"{ticker} — Forecasted Buy/Sell Zones\")\n \n        st.plotly_chart(fig, use_container_width=True)\n \n        with st.expander(\"■ Forecasted Opportunities\"):\n-            c1, c2 = st.columns(2)\n-            c1.markdown(\"#### ■ Future Buy Opportunities\")\n+            buy_col, sell_col = st.columns(2)\n+\n+            buy_col.markdown(\"#### ■ Future Buy Opportunities\")\n \n            if buy_future is None or buy_future.empty:\n-                c1.info(\"No future buy signals detected ■\")\n+                buy_col.info(\"No buy signals ahead ■\")\n            else:\n-                c1.dataframe(buy_future[[\"trade_date\", col_close]], use_container_width=True)\n-\n-            c2.markdown(\"#### ■ Future Sell Opportunities\")\n+                buy_col.dataframe(buy_future[[\"trade_date\", col_close]],\n+                                  use_container_width=True)\n+\n+            sell_col.markdown(\"#### ■ Future Sell Opportunities\")\n \n            if sell_future is None or sell_future.empty:\n-                c2.info(\"No future sell signals detected ■\")\n+                sell_col.info(\"No sell signals detected ■\")\n            else:\n-                c2.dataframe(sell_future[[\"trade_date\", col_close]], use_container_width=True)\n+                sell_col.dataframe(sell_future[[\"trade_date\", col_close]],\n+                                   use_container_width=True)\n+\n+\n+\n# ============================================================\n# TAB 6 — DWM: DATA EXPLORATION & PREPROCESSING\n+# Syllabus: KDD, attribute types, statistical description,\n+# visualization, cleaning, transformation, reduction.\n+# ============================================================\n+with tab6:\n+    st.subheader(\"■ Data Exploration & Preprocessing (DWM Module 3)\")\n+    st.caption(\"Uses the selected stock's stored OHLCV data. This is an educational DWM workflow, not just a stock indicator view.\")\n+    eda_ticker = st.selectbox(\"Ticker for EDA\", selected_companies, key=\"dwm_eda_ticker\")\n+    raw_eda = fetch_prices(eda_ticker, start_date, end_date)\n+    if raw_eda is None or raw_eda.empty:\n+        st.warning(\"No rows are available for this ticker/date range.\")\n+    else:\n+        raw_summary = summarize_dwm_dataset(raw_eda)\n+        cleaned_eda = prepare_dwm_dataset(raw_eda)\n+        clean_summary = summarize_dwm_dataset(cleaned_eda)\n+        c1, c2, c3 = st.columns(3)\n+        c1.metric(\"Raw Rows\", f\"{len(raw_eda):,}\")\n+        c2.metric(\"Cleaned Rows\", f\"{len(cleaned_eda):,}\")\n+        c3.metric(\"Derived Attributes\", f\"{max(0, cleaned_eda.shape[1] - raw_eda.shape[1])}\")\n+        st.markdown(\"#### Dataset quality before preprocessing\")\n+        st.dataframe(raw_summary[\"overview\"], use_container_width=True, hide_index=True)\n+        st.markdown(\"#### Missing values and attribute types (cleaned dataset)\")\n+        st.dataframe(clean_summary[\"missing\"], use_container_width=True, hide_index=True)\n+        st.markdown(\"#### Statistical description\")\n+        st.dataframe(clean_summary[\"statistics\"], use_container_width=True, hide_index=True)\n+        numeric_cols = cleaned_eda.select_dtypes(include=np.number).columns.tolist()\n+        default_col = \"close\" if \"close\" in numeric_cols else numeric_cols[0]\n+        chart_col = st.selectbox(\"Numeric attribute to visualize\", numeric_cols, index=numeric_cols.index(default_col), key=\"dwm_eda_chart_col\")\n+        chart_kind = st.radio(\"Visualization\", [\"Line chart\", \"Histogram\", \"Box plot\"], horizontal=True, key=\"dwm_eda_chart_kind\")\n+        if chart_kind == \"Line chart\" and \"trade_date\" in cleaned_eda:\n+            fig = px.line(cleaned_eda, x=\"trade_date\", y=chart_col, title=f\"{chart_col} over time\")\n+        elif chart_kind == \"Histogram\":\n+            fig = px.histogram(cleaned_eda, x=chart_col, title=f\"Distribution of {chart_col}\")\n+        else:\n+            fig = px.box(cleaned_eda, y=chart_col, points=\"outliers\", title=f\"Box plot of {chart_col}\")\n+        st.plotly_chart(fig, use_container_width=True)\n+        with st.expander(\"View preprocessed rows\"):\n+            st.dataframe(cleaned_eda, use_container_width=True)\n+        download_csv(cleaned_eda, f\"{eda_ticker}_dwm_preprocessed\")\n+        st.info(\"Cleaning applied: date sorting, duplicate-date removal, numeric conversion, invalid close removal, non-finite handling, and median imputation for numeric features. Derived columns include returns, SMA/EMA, volatility, and next-day targets.\")\n+\n+\n+\n# ============================================================\n# TAB 7 — DWM: CLASSIFICATION & REGRESSION\n+# Syllabus: Decision Tree, Naive Bayes, simple/multiple regression.\n+# ============================================================\n+with tab7:\n+    st.subheader(\"■ Classification & Regression (DWM Module 4)\")\n+    st.markdown(\"Predict **next-day direction** with Decision Tree and Naive Bayes, and estimate **next-day closing price** with multiple linear regression. Evaluation uses a chronological train/test split.\")\n+    ml_ticker = st.selectbox(\"Ticker for supervised mining\", selected_companies, key=\"dwm_ml_ticker\")\n+    ml_raw = fetch_prices(ml_ticker, start_date, end_date)\n+    if ml_raw is None or ml_raw.empty:\n+        st.warning(\"No data is available for supervised learning.\")\n+    else:\n+        try:\n+            with st.spinner(\"Training DWM demonstration models...\"):\n+                ml_result = run_dwm_supervised_analysis(ml_raw)\n+            st.markdown(\"#### Regression — next-day closing price\")\n+            st.dataframe(ml_result[\"regression_metrics\"], use_container_width=True, hide_index=True)\n+            pred_df = ml_result[\"regression_predictions\"]\n+            fig = go.Figure()\n+            fig.add_trace(go.Scatter(x=pred_df[\"trade_date\"], y=pred_df[\"Actual Next Close\"], mode=\"lines\", name=\"Actual\"))\n+            fig.add_trace(go.Scatter(x=pred_df[\"trade_date\"], y=pred_df[\"Predicted Next Close\"], mode=\"lines\", name=\"Predicted\"))\n+            fig.update_layout(title=\"Actual vs Predicted Next Close\", xaxis_title=\"Date\", yaxis_title=\"Price\")\n+            st.plotly_chart(fig, use_container_width=True)\n+            st.markdown(\"#### Classification — next-day direction\")\n+            st.dataframe(ml_result[\"classification_metrics\"], use_container_width=True, hide_index=True)\n+            st.caption(f\"Chronological split: {ml_result['train_rows']:,} training rows / {ml_result['test_rows']:,} testing rows. Features: {', '.join(ml_result['feature_columns'])}.\")\n+            download_csv(ml_result[\"classification_metrics\"], f\"{ml_ticker}_classification_metrics\")\n+            download_csv(pred_df, f\"{ml_ticker}_regression_predictions\")\n+            st.warning(\"These are historical hold-out metrics, not a promise of future accuracy. The Decision Tree uses entropy (ID3-style); it is not a complete from-scratch implementation of textbook ID3.\")\n+        except Exception as exc:\n+            st.error(f\"Could not run supervised analysis: {exc}\")\n+\n+\n+\n# ============================================================\n# TAB 8 — DWM: CLUSTERING\n+# Syllabus: K-Means and hierarchical/agglomerative clustering.\n+# ============================================================\n+with tab8:\n+    st.subheader(\"■ Stock Profile Clustering (DWM Module 5)\")\n+    st.write(\"Group selected companies using return, volatility, volume, and price-range features. Features are standardized before clustering.\")\n+    cluster_method = st.selectbox(\"Clustering method\", [\"K-Means\", \"Hierarchical (Agglomerative)\"], key=\"dwm_cluster_method\")\n+    if len(selected_companies) < 2:\n+        cluster_count = 2\n+        st.info(\"Select at least two companies in the sidebar to compare clusters.\")\n+    else:\n+        max_clusters = min(6, len(selected_companies))\n+        if max_clusters == 2:\n+            cluster_count = 2\n+            st.caption(\"Two companies selected; clustering will use two clusters.\")\n+        else:\n+            cluster_count = st.slider(\"Number of clusters\", 2, max_clusters, min(3, max_clusters), key=\"dwm_cluster_count\")\n+    history_map = {ticker: fetch_prices(ticker, start_date, end_date) for ticker in selected_companies}\n+    try:\n+        profiles = build_stock_profiles(history_map)\n+        clustered = cluster_stock_profiles(profiles, cluster_count, cluster_method)\n+        st.dataframe(clustered, use_container_width=True, hide_index=True)\n+        if not clustered.empty:\n+            fig = px.scatter(clustered, x=\"Average Daily Return %\", y
```

=\"Return Volatility %\", color=clustered[\"Cluster\"].astype(str), hover_name=\"Ticker\", size=\"Price Range %\", title=f\"{cluster_method} — Return vs Volatility\")\n+ fig.update_layout(legend_title=\"Cluster\")\n+ st.plotly_chart(fig, use_container_width=True)\n+ st.markdown(\"#### Cluster interpretation\")\n+ interpretation = clustered.groupby(\"Cluster\").agg(Companies=(\"Ticker\", lambda s: \", \".join(s)), Mean_Return_Pct=(\"Average Daily Return %\", \"mean\"), Mean_Volatility_Pct=(\"Return Volatility %\", \"mean\"), Mean_Total_Return_Pct=(\"Total Return %\", \"mean\")).reset_index().round(3)\n+ st.dataframe(interpretation, use_container_width=True, hide_index=True)\n+ download_csv(clustered, \"dwm_stock_clusters\")\n+ except Exception as exc:\n+ st.error(f\"Clustering could not run: {exc}\")\n+\n+\n+# ==\n+# TAB 9 — DWM: FREQUENT PATTERNS & ASSOCIATION RULES\n+# Syllabus: frequent itemsets, support, confidence, lift, Apriori.\n+# ==\n+with tab9:\n+ st.subheader(\"■ Frequent Pattern Mining — Apriori (DWM Module 6)\")\n+ st.write(\"Each trading date is treated as a transaction containing selected ticker states (UP/DOWN/FLAT, HIGH/LOW_VOLUME, HIGH/LOW_MOVE). Rules describe historical co-occurrence, not causation.\")\n+ support_pct = st.slider(\"Minimum support (%)\", 1, 60, 10, key=\"dwm_apriori_support\")\n+ confidence_pct = st.slider(\"Minimum confidence (%)\", 10, 100, 50, key=\"dwm_apriori_confidence\")\n+ max_items = st.select_slider(\"Maximum itemset size\", options=[2, 3], value=3, key=\"dwm_apriori_size\")\n+ pattern_histories = {ticker: fetch_prices(ticker, start_date, end_date) for ticker in selected_companies}\n+ transactions = build_market_transactions(pattern_histories)\n+ st.metric(\"Daily transactions\", f\"{len(transactions):,}\")\n+ if not transactions:\n+ st.info(\"No transactions could be built for the selected date range.\")\n+ else:\n+ itemsets, rules = mine_association_rules(transactions, support_pct / 100, confidence_pct / 100, max_items)\n+ if itemsets.empty:\n+ st.info(\"No frequent itemsets meet the selected support threshold. Lower minimum support and retry.\")\n+ else:\n+ st.markdown(\"#### Frequent itemsets\")\n+ st.dataframe(itemsets, use_container_width=True, hide_index=True)\n+ download_csv(itemsets, \"dwm_frequent_itemsets\")\n+ st.markdown(\"#### Association rules\")\n+ if rules.empty:\n+ st.info(\"No rules meet both thresholds. Lower support/confidence or select more tickers.\")\n+ else:\n+ display_rules = rules.copy()\n+ for col in [\"Support\", \"Confidence\"]:\n+ display_rules[col] = (display_rules[col] * 100).round(2).astype(str) + \"%\"\n+ display_rules[\"Lift\"] = display_rules[\"Lift\"].round(3)\n+ st.dataframe(display_rules, use_container_width=True, hide_index=True)\n+ download_csv(rules, \"dwm_association_rules\")\n+ st.caption(\"Support = fraction of trading dates containing both sides. Confidence = frequency of the consequent when the antecedent occurs. Lift > 1 indicates a positive association in this sample.\")\n+\n+\n+# ==\n+# TAB 10 — DWM: OLAP-STYLE ANALYSIS\n+# Syllabus connection: warehouse facts, aggregation, roll-up, drill-down,\n+# slice and dice. This operates on the available stock-price fact data.\n+# ==\n+with tab10:\n+ st.subheader(\"■ OLAP Cube — Slice, Dice, Roll-up & Drill-down\")\n+ st.write(\"Explore stored stock facts by ticker and time. Yearly aggregation is a roll-up; quarterly/monthly views drill down; ticker and direction filters perform slice/dice.\")\n+ olap_histories = {ticker: fetch_prices(ticker, start_date, end_date) for ticker in selected_companies}\n+ fact_table = build_olap_fact_table(olap_histories)\n+ if fact_table.empty:\n+ st.warning(\"No fact rows are available for OLAP aggregation.\")\n+ else:\n+ col_a, col_b, col_c = st.columns(3)\n+ with col_a:\n+ olap_level = st.selectbox(\"Time granularity\", [\"Yearly\", \"Quarterly\", \"Monthly\"], index=2, key=\"dwm_olap_level\")\n+ with col_b:\n+ olap_tickers = st.multiselect(\"Slice by ticker\", selected_companies, default=selected_companies, key=\"dwm_olap_tickers\")\n+ with col_c:\n+ olap_direction = st.selectbox(\"Dice by direction\", [\"All\", \"Up\", \"Down\", \"Flat\"], key=\"dwm_olap_direction\")\n+ olap_result = olap_aggregate(fact_table, olap_level, olap_tickers, olap_direction)\n+ if olap_result.empty:\n+ st.info(\"No rows match the current OLAP filters.\")\n+ else:\n+ st.dataframe(olap_result, use_container_width=True, hide_index=True)\n+ st.markdown(\"#### Average close by period\")\n+ period_cols = [c for c in [\"Year\", \"Quarter\", \"Month\"] if c in olap_result.columns]\n+ olap_result[\"Period\"] = olap_result[period_cols].astype(str).agg(\"-\".join, axis=1)\n+ fig = px.bar(olap_result, x=\"Period\", y=\"Average_Close\", color=\"Ticker\", barmode=\"group\", title=\"Average close by OLAP period\")\n+ st.plotly_chart(fig, use_container_width=True)\n+ download_csv(olap_result, \"dwm_olap_aggregate\")\n+ with st.expander(\"View underlying fact table\"):\n+ st.dataframe(fact_table, use_container_width=True, hide_index=True)\n+ download_csv(fact_table, \"dwm_stock_fact_table\")\n"

}

4. AI Review

{

"summary": "The proposed code update significantly expands `Stocks_predictor/app.py` by integrating advanced Data Warehousing and Mining (DWM) functions from `calculations.py`, adding 5 new dedicated tabs (Tabs 6–10) covering data exploration, preprocessing, classification, regression, clustering, frequent pattern mining, and OLAP cube aggregations. It also refactors the UI layout, updates metadata, adds a comprehensive sector mapping, and enhances technical indicator displays in Tab 1.",

"findings": [

{

"finding_id": "change-review-1",

"severity": "medium",

"title": "Uncaught Exceptions in Imported DWM Analytics Functions",

"explanation": "The newly added tabs (Tabs 6 through 10) invoke complex calculations from `calculations.py` (e.g., `run_dwm_supervised_analysis`, `cluster_stock_profiles`, `mine_association_rules`) without robust null-checks or fallback mechanisms around dataframe operations, which could cause the Streamlit application to crash or display uncaught exceptions if intermediate dataframes are empty or malformed.",

"file_path": "Stocks_predictor/app.py",

"line_number": 860,

"recommendation": "Wrap complex DWM computations in try-except blocks with user-friendly Streamlit error messages and graceful fallbacks.",

"confidence": 0.85,

"metadata": {}

},

{

"finding_id": "change-review-2",

"severity": "low",

"title": "Expanded Scope and Maintainability",

"explanation": "While the developer reason states the change includes calculations from `calculations.py` to fix frontend errors, the commit vastly increases the file length (introducing extensive sector mappings, brand headers, and 5 complex DWM tabs). This blurs the separation of concerns and reduces maintainability of a single monolithic `app.py` file.",

"file_path": "Stocks_predictor/app.py",

"line_number": 1,

"recommendation": "Refactor the UI code by moving individual tab logic and large static definitions (like `sector_map`) into separate modular component s or views.",

"confidence": 0.9,

"metadata": {}

},

{

"finding_id": "change-review-3",

"severity": "info",

"title": "Successful Integration of DWM Functionality",

"explanation": "The proposal successfully satisfies the developer reason by importing and invoking the newly implemented functions from `calculations.p y` and `data_fetcher.py` (`get_latest_date`, `add_technical_indicators`, DWM modules), resolving frontend errors and successfully rendering the requested a nalytical views.",

"file_path": "Stocks_predictor/app.py",

"line_number": 15,

"recommendation": "Ensure comprehensive end-to-end integration testing is performed on all new tabs to confirm data flows correctly from the database through calculations to the Streamlit UI.",

"confidence": 0.95,

"metadata": {}

}

],

"provider": "gemini",

"model": "gemini-3.5-flash-lite",

"raw_response": "{\n \"summary\": \"The proposed code update significantly expands `Stocks_predictor/app.py` by integrating advanced Data Warehousin g and Mining (DWM) functions from `calculations.py`, adding 5 new dedicated tabs (Tabs 6–10) covering data exploration, preprocessing, classification, regr ession, clustering, frequent pattern mining, and OLAP cube aggregations. It also refactors the UI layout, updates metadata, adds a comprehensive sector m apping, and enhances technical indicator displays in Tab 1.\",\n \"findings\": [\n {\n \"finding_id\": \"change-review-1\",\n \"severity\": \"medium\",\n \"title\": \"Uncaught Exceptions in Imported DWM Analytics Functions\",\n \"explanation\": \"The newly added tabs (Tabs 6 through 10) invoke complex ca lculations from `calculations.py` (e.g., `run_dwm_supervised_analysis`, `cluster_stock_profiles`, `mine_association_rules`) without robust null-checks or fallb ack mechanisms around dataframe operations, which could cause the Streamlit application to crash or display uncaught exceptions if intermediate datafram es are empty or malformed.\",\n \"file_path\": \"Stocks_predictor/app.py\",\n \"line_number\": 860,\n \"recommendation\": \"Wrap complex DWM com putations in try-except blocks with user-friendly Streamlit error messages and graceful fallbacks.\",\n \"confidence\": 0.85\n },\n {\n \"finding_id\": \"c hange-review-2\",\n \"severity\": \"low\",\n \"title\": \"Expanded Scope and Maintainability\",\n \"explanation\": \"While the developer reason states the change includes calculations from `calculations.py` to fix frontend errors, the commit vastly increases the file length (introducing extensive sector mappings, brand headers, and 5 complex DWM tabs). This blurs the separation of concerns and reduces maintainability of a single monolithic `app.py` file.\",\n \"file _path\": \"Stocks_predictor/app.py\",\n \"line_number\": 1,\n \"recommendation\": \"Refactor the UI code by moving individual tab logic and large static definitions (like `sector_map`) into separate modular components or views.\",\n \"confidence\": 0.90\n },\n {\n \"finding_id\": \"change-review-3\",\n \"severity\": \"info\",\n \"title\": \"Successful Integration of DWM Functionality\",\n \"explanation\": \"The proposal successfully satisfies the developer r eason by importing and invoking the newly implemented functions from `calculations.py` and `data_fetcher.py` (`get_latest_date`, `add_technical_indicators` , DWM modules), resolving frontend errors and successfully rendering the requested analytical views.\",\n \"file_path\": \"Stocks_predictor/app.py\",\n \ "line_number\": 15,\n \"recommendation\": \"Ensure comprehensive end-to-end integration testing is performed on all new tabs to confirm data flows corre ctly from the database through calculations to the Streamlit UI.\",\n \"confidence\": 0.95\n }\n]\n}"

}

5. AI Remediation

{

"finding_id": "developer-change",

"file_path": "Stocks_predictor/app.py",

"explanation": "The developer proposal successfully integrates advanced Data Warehousing and Mining (DWM) functions from `calculations.py` into `Stock s_predictor/app.py`, adding Tabs 6 through 10 as well as enhanced analytics, sector maps, and technical indicators in Tab 1. To address the medium-severi ty finding regarding uncaught exceptions in complex DWM analytics functions (`run_dwm_supervised_analysis`, `cluster_stock_profiles`, `mine_association_ rules`, etc.), we wrap the computations in robust `try-except` blocks with user-friendly Streamlit error messages and graceful fallbacks, ensuring stability and preventing crashes on empty or malformed intermediate DataFrames.",

"original_code": "import streamlit as st\nimport pandas as pd\nimport numpy as np\nfrom datetime import datetime, date\nimport plotly.graph_objects as go \nimport plotly.express as px\n\nfrom calculations import (\n fetch_prices, fetch_current_price, fetch_company_info,\n compute_sma, compute_ema, det ect_abrupt_changes,\n volatility_and_risk, correlation_analysis,\n compare_companies, get_close_price_column\n)\nfrom plotting import plot_correlation \nfrom data_fetcher import get_company_list, run_fetching\n\n\n# ============== BASIC PAGE CONFIG ==============...",

"replacement_code": "import streamlit as st\nimport pandas as pd\nimport numpy as np\nfrom datetime import datetime, date\nimport plotly.graph_objects as go\nimport plotly.express as px\n\nfrom plotting import plot_correlation\nfrom calculations import (\n fetch_prices, fetch_current_price, fetch_company_i nfo,\n compute_sma, compute_ema, detect_abrupt_changes,\n volatility_and_risk, correlation_analysis,\n compare_companies, get_close_price_colu

```
mn,\n    add_technical_indicators,\n    prepare_dwm_dataset, summarize_dwm_dataset,\n    run_dwm_supervised_analysis, build_stock_profiles,\n    cluster
_stock_profiles, build_market_transactions,\n    mine_association_rules, build_olap_fact_table, olap_aggregate,\n)\nfrom data_fetcher import get_company_
list, run_fetching, get_latest_date\n\nselected_companies = []\n\nsector_map = {\n    \"RELIANCE.NS\":  \"Energy & Petrochemicals ■■\",\n    \"HDFCBANK
.NS\":  \"Banking & Finance ■\",\n    \"TCS.NS\":  \"IT & Technology ■\",\n    \"BHARTIARTL.NS\": \"Telecom ■\",\n    \"ICICIBANK.NS\": \"Banking & Fi
nance ■\",\n    \"SBIN.NS\":  \"Banking & Finance ■\",\n    \"INFY.NS\":  \"IT & Technology ■\",\n    \"HINDUNILVR.NS\": \"Consumer Goods ■\",\n    \"
LICI.NS\":  \"Insurance ■■\",\n    \"BAJFINANCE.NS\": \"Financial Services ■\",\n    \"ITC.NS\":  \"Diversified FMCG ■\",\n    \"LT.NS\":  \"Infrastru
cture & Engineering ■■\",\n    \"MARUTI.NS\":  \"Automobiles ■\",\n    \"HCLTECH.NS\":  \"IT & Technology ■\",\n    \"SUNPHARMA.NS\":  \"Pharmaceuti
cals ■\",\n    \"KOTAKBANK.NS\":  \"Banking & Finance ■\",\n    \"M&M.NS\":  \"Automobiles, Mobility ■\",\n    \"ULTRACEMCO.NS\": \"Cement & Buildin
g Material ■\",\n    \"AXISBANK.NS\":  \"Banking & Finance ■\",\n    \"NTPC.NS\":  \"Power Generation ■\",\n    \"TITAN.NS\":  \"Luxury & Retail ■■\",\
n    \"BAJAJFINSV.NS\": \"Financial Services ■\",\n    \"HAL.NS\":  \"Defence & Aerospace ✈■\",\n    \"ONGC.NS\":  \"Oil & Gas ■\",\n    \"ADANIPOR
TS.NS\": \"Logistics & Ports ■\",\n    \"BEL.NS\":  \"Defence Electronics ■■\",\n    \"WIPRO.NS\":  \"IT & Technology ■\",\n    \"JSWSTEEL.NS\":  \"Me
tals & Steel ■\",\n    \"TATAMOTORS.NS\": \"Automobiles ■\",\n    \"ASIANPAINT.NS\": \"Consumer Goods ■\",\n    \"COALINDIA.NS\": \"Mining & Commo
dities ■■\",\n    \"NESTLEIND.NS\":  \"Food & Beverages ■\",\n    \"GRASIM.NS\":  \"Materials & Chemicals ■\",\n    \"HINDALCO.NS\": \"Metals & Mining
■■\",\n    \"TATASTEEL.NS\":  \"Steel Manufacturing ■\",\n    \"AMBUJACEM.NS\": \"Cement & Construction ■\",\n    \"RPOWER.NS\":  \"Power Generati
on ■\",\n    \"DIVISLAB.NS\":  \"Pharmaceuticals ■\",\n    \"SHREECEM.NS\":  \"Cement & Building Material ■\",\n    \"ADANIENT.NS\":  \"Diversified ■\"\n}
\n\n# ============== BASIC PAGE CONFIG ==============\nst.set_page_config(\n    page_title=\"Stock Insights – Smart Nifty50 Analytics\",\n    pag
e_icon=\"■\",\n    layout=\"wide\",\n    initial_sidebar_state=\"expanded\",\n)\n\n# ============== THEME TOGGLE ==============\nif \"theme\" not in
st.session_state:\n    st.session_state[\"theme\"] = \"Dark\"\n\ntheme = st.sidebar.radio(\n    \"Theme\",\n    [\"Dark\", \"Light\"],\n    index=0 if st.session_state[
\"theme\"] == \"Dark\" else 1,\n)\nst.session_state[\"theme\"] = theme\n\nif theme == \"Dark\":\n    bg_color     = \"#060814\"\n    card_color   = \"#101623\"\n
    text_color   = \"#fafafa\"\n    subtext_color = \"rgba(122, 234, 255, 0.85)\"\n    accent       = \"#00e1ff\"\nelse:\n    bg_color     = \"#f3f8ff\"\n    card_color
= \"#ffffff\"\n    text_color   = \"#111827\"\n    subtext_color = \"#444\"\n    accent       = \"#0077ff\"\n\n# ============== SEO META TAGS ==========
===\nst.markdown(\"\"\"\n    <meta name=\"description\" content=\"AI-powered stock learning platform — Master Nifty50 analysis with SMA, RSI, volatility, c
orrelation, and smart insights.\">\n    <meta name=\"keywords\" content=\"Stock Market, Nifty50, Finance Learning, Technical Indicators, SMA, RSI, MACD,
Risk & Volatility, AI Stock Analysis\">\n    <meta property=\"og:title\" content=\"Stock Insights — Master the Market Step-by-Step\">\n    <meta property=\"og:
description\" content=\"Smart Insights for Smarter Investing — Nifty50 analytics powered by AI.\">\n    <meta property=\"og:type\" content=\"website\">\n\"\"\"
, unsafe_allow_html=True)\n\n# ============== BRAND HEADER ==============\nst.markdown(f\"\"\"\n    <style>\n        .brand-title {{\n            font-si
ze: 42px;\n            font-weight: 900;\n            background: linear-gradient(90deg, #12c2e9, #0ee6b7, #00ff95);\n            -webkit-background-clip: text;\n
-webkit-text-fill-color: transparent;\n            text-transform: uppercase;\n            letter-spacing: 4px;\n            margin-bottom: 4px;\n        }}\n        .brand-tagli
ne {{\n            font-size: 24px;\n            font-weight: 700;\n            color: {text_color};\n            margin-top: 0px;\n            margin-bottom: 6px;\n        }}\n        .br
and-sub {{\n            font-size: 15px;\n            font-weight: 500;\n            color: {subtext_color};\n            margin-top: 0px;\n        }}\n    </style>\n\n    <div style=
\"text-align:center; margin-top:10px; margin-bottom:30px;\">\n        <div class=\"brand-title\">Stock Insights</div>\n        <div class=\"brand-tagline\">Master t
he Market Step-by-Step</div>\n        <div class=\"brand-sub\">Nifty50 Analytics — Smart Insights for Smarter Investing</div>\n    </div>\n\"\"\", unsafe_allow
_html=True)\n\n# ============== AUTO UPDATE ==============\n@st.cache_resource(ttl=24 * 60 * 60)\ndef silent_update():\n    run_fetching()\n\nw
ith st.spinner(\"Syncing latest stock data…\"):\n    silent_update()\n\n# ============== SIDEBAR FILTERS ==============\ncompany_list = get_comp
any_list()\nif not company_list:\n    st.error(\"■ No tickers found in DB — run insert_companies first!\")\n    st.stop()\n\nselected_companies = st.sidebar.multi
select(\n    \"Select Company Tickers\",\n    options=company_list,\n    default=company_list[:2],\n)\n\nif not selected_companies:\n    st.warning(\"Select at l
east one company.\")\n    st.stop()\n\nMIN_DATE = date(2015, 1, 1)\nUI_MAX_DATE = date(date.today().year, 12, 31)\n\nstart_date = st.sidebar.date_inp
ut(\n    \"Start Date\",\n    value=MIN_DATE,\n    min_value=MIN_DATE,\n    max_value=UI_MAX_DATE,\n)\n\nend_date = st.sidebar.date_input(\n    \"End
Date\",\n    value=date.today(),\n    min_value=MIN_DATE,\n    max_value=UI_MAX_DATE,\n)\n\nif start_date >= end_date:\n    st.sidebar.error(\"Start date
must be earlier than End Date.\")\n    st.stop()\n\nTODAY = date.today()\nif end_date > TODAY:\n    end_date = TODAY\n\ndate_valid = True\n\n# ---- Stale
ness check ----\nlast_updated_dates = []\nfor ticker in selected_companies:\n    d = get_latest_date(ticker)\n    if d:\n        last_updated_dates.append(d)\n\nl
ast_updated = (\n    max(last_updated_dates).strftime(\"%d %b %Y\")\n    if last_updated_dates\n    else \"Unknown\")\n\nstale_msgs = []\ntoday = datetim
e.now().date()\nfor t in selected_companies:\n    d = get_latest_date(t)\n    if d:\n        delay = (today - d).days\n        if delay >= 2:\n            icon = \"■\" if dela
y > 5 else \"■■\"\n            stale_msgs.append(f\"{icon} <strong>{t}</strong> — {delay} days ago\")\n\nif stale_msgs:\n    msg_html = \"<br>\".join(stale_msgs
)\n    st.sidebar.markdown(f\"\"\"\n    <div style=\"\n        background:rgba(255,180,0,0.12);\n        border-left:4px solid #ffae00;\n        padding:12px 14px;\n
border-radius:8px;\n        margin-top:20px;\n        color:#ffe7c4;\">\n        <strong>■ Update Delay Detected</strong><br>\n        <span style='font-size:13
px;'>{msg_html}</span>\n    </div>\n    \"\"\", unsafe_allow_html=True)\nelse:\n    st.sidebar.markdown(f\"\"\"\n    <div style=\"\n        background:rgba(0,255,
140,0.10);\n        border-left:4px solid #00ff9d;\n        padding:12px 14px;\n        border-radius:8px;\n        margin-top:20px;\n        color:#baffdd;\">\n        ■ Al
l selected stocks are updated till the latest available date!\n    </div>\n    \"\"\", unsafe_allow_html=True)\n\n# ============== FOOTER (Updated daily, no
developer name) ==============\nst.markdown(\n    f\"\"\"\n    <div style=\"\n        text-align:center;\n        padding:10px;\n        margin-top:30px;\n        fo
nt-size:14px;\n        font-weight:500;\n        color:#a6fff5;\n        border-top:1px solid rgba(255,255,255,0.15);\n    \">\n        <span style='font-size:13px; color:
#9cdcff;'>\n            ■ Updated daily at <strong>10:00 AM</strong> & <strong>11:00 PM IST</strong><br>\n            ■ Data available till: <strong style='color
:#7afcff;'>{last_updated}</strong>\n        </span>\n    </div>\n    \"\"\",\n    unsafe_allow_html=True,\n)\n\nif selected_companies:\n    st.sidebar.markdown(\"\
\"\n    <div style=\"\n        background:rgba(0,255,200,0.06);\n        border-left:4px solid #00ffc8;\n        padding:10px 14px;\n        border-radius:6px;\n        m
argin-top:14px;\n        color:#b9fff4;\n    \">\n    <strong>■■ Selected Stocks:</strong>\n    </div>\n    \"\"\", unsafe_allow_html=True)\n\n    for t in selected_c
ompanies:\n        sector = sector_map.get(t, \"Sector Not Available ■■\")\n        st.sidebar.markdown(\n            f\"<span style='color:#d9f7ff;'>• <strong>{t}</s
trong> — {sector}</span>\",\n            unsafe_allow_html=True,\n        )\n\nst.sidebar.markdown(\"\"\"\n<div style=\"\n    background:rgba(0,255,200,0.06);\n
border-left:4px solid #00ffc8;\n    padding:10px 14px;\n    border-radius:6px;\n    margin-top:14px;\n    color:#b9fff4;\n\">\n<strong>■ Date Range:</strong>
<br>\n\"\"\", unsafe_allow_html=True)\n\nst.sidebar.markdown(\n    f\"<span style='color:#d9f7ff;'>• From: <strong>{start_date}</strong></span><br>\"\n    f\"
<span style='color:#d9f7ff;'>• To:  <strong>{end_date}</strong></span>\",\n    unsafe_allow_html=True,\n)\nst.sidebar.markdown(\"</div>\", unsafe_allow_h
tml=True)\n\n\n# ============== Helper Functions ==============\ndef build_budget_options():\n    small = list(range(0, 100001, 10000))\n    large =
list(range(200000, 1000001, 100000))\n    values = small + large\n    labels = [f\"■{v:,.0f}\" for v in values]\n    return values, labels\n\n\ndef analyze_trend_c
onfidence(df, col_close, horizon):\n    if len(df) < 15:\n        return 50, \"Hold\", 0, 0\n\n    lookback = 60 if horizon == \"Short Term\" else 180\n    recent   = df.t
ail(min(len(df), lookback))\n\n    x = np.arange(len(recent))\n    y = recent[col_close].values\n    slope, _ = np.polyfit(x, y, 1)\n    pct_change = (y[-
1] - y[0]) / y[0] * 100\n    vol = recent[col_close].pct_change().std() * 100\n\n    conf = 50 + pct_change / 2 - vol / 4\n    conf = float(max(5, min(95, conf)))\
n\n    if pct_change > 5 and slope > 0:\n        label = \"Strong Buy\"\n    elif pct_change > 1 and slope > 0:\n        label = \"Buy\"\n    elif pct_change < -5 and sl
ope < 0:\n        label = \"Risky / Avoid\"\n    else:\n        label = \"Hold\"\n\n    return conf, label, pct_change, vol\n\n\ndef project_future(df, col_close, horizon):
\n    lookback = 60 if horizon == \"Short Term\" else 180\n    if len(df) < lookback:\n        lookback = len(df)\n\n    recent = df.tail(lookback)\n    if len(recent) <
10:\n        return pd.DataFrame(), pd.DataFrame()\n\n    x = np.arange(len(recent))\n    y = recent[col_close].values\n    slope, intercept =
np.polyfit(x, y, 1)\n\n    future_days = 15 if horizon == \"Short Term\" else 60\n    last_date = df[\"trade_date\"].iloc[-1]\n    future_dates = pd.bdate_range(
last_date + pd.Timedelta(days=1), periods=future_days)\n    x_future = np.arange(len(recent), len(recent) + len(future_dates))\n    future_prices = interce
pt + slope * x_future\n\n    future_df = pd.DataFrame({\"trade_date\": future_dates, col_close: future_prices})\n    future_df[\"SMA\"] = future_df[col_clo
se].rolling(20, min_periods=1).mean()\n\n    buy = future_df[future_df[col_close] < future_df[\"SMA\"]]\n    sell = future_df[future_df[col_close] > future_df[\"S
MA\"]]\n    return buy, sell\n\n\ndef download_csv(df: pd.DataFrame, name: str):\n    \"\"\"Reusable download helper for tabular data.\"\"\"\n    st.download_bu
tton(\n        label=\"■ Download this data as CSV\",\n        data=df.to_csv(index=False),\n        file_name=f\"{name}.csv\",\n        mime=\"text/csv\",\n    )\n\n\n
# ============== TABS ==============\ntab1, tab2, tab3, tab4, tab5, tab6, tab7, tab8, tab9, tab10 = st.tabs([\n    \"■ Price Trends\",\n    \"■ Abrupt C
hanges\",\n    \"■ Risk & Volatility\",\n    \"■ Compare & Correlate\",\n    \"■ Smart Insights\",\n    \"■ DWM: Explore & Clean\",\n    \"■ DWM: Classification &
Regression\",\n    \"■ DWM: Clustering\",\n    \"■ DWM: Apriori Rules\",\n    \"■ DWM: OLAP Cube\",\n])\n\n\n# ============== TAB 1 — Price Trends + I
ndicators ==============\nwith tab1:\n    st.subheader(\"■ Price Trend + Technical Indicators\")\n    st.markdown(\"### ■ Learn & Analyze with Price Tre
```

```
nds\")\n\n    col1, col2 = st.columns(2)\n    with col1:\n        st.markdown(\"\"\"\n        <div style=\"\n            background:rgba(0,180,216,0.08);\n            border-left:4px solid #00b4d8;\n            padding:12px 16px;\n            border-radius:8px;\n            color:#c9e8ff;\n            margin-bottom:12px;\n        \">\n        <strong>■ What will you analyze here?</strong><br>\n        • Stock price behaviour & market cycles<br>\n        • Spot big uptrends & downtrends<br>\n        • Understand momentum using SMA/EMA<br>\n        • Validate trend direction confidently\n        </div>\n        \"\"\", unsafe_allow_html=True)\n\n    with col2:\n        st.markdown(\"\"\"\n        <div style=\"\n            background:rgba(0,180,216,0.08);\n            border-left:4px solid #00b4d8;\n            padding:12px 16px;\n            border-radius:8px;\n            color:#c9e8ff;\n            margin-bottom:12px;\n        \">\n        <strong>■ RSI — Momentum Pressure</strong><br>\n        • RSI > 70 = Overbought → Possible correction<br>\n        • RSI < 30 = Oversold → Possible bounce<br>\n        • Helps time better entry & exit points\n        </div>\n        \"\"\", unsafe_allow_html=True)\n\n    col3, col4 = st.columns(2)\n    with col3:\n        st.markdown(\"\"\"\n        <div style=\"\n            background:rgba(0,180,216,0.08);\n            border-left:4px solid #00b4d8;\n            padding:12px 16px;\n            border-radius:8px;\n            color:#c9e8ff;\n            margin-bottom:12px;\n        \">\n        <strong>■ SMA / EMA — Trend Direction</strong><br>\n        • Price above MA → Bullish momentum<br>\n        • Price below MA → Bearish weakness<br>\n        • Short SMAs react faster to volatility\n        </div>\n        \"\"\", unsafe_allow_html=True)\n\n    with col4:\n        st.markdown(\"\"\"\n        <div style=\"\n            background:rgba(0,180,216,0.08);\n            border-left:4px solid #00b4d8;\n            padding:12px 16px;\n            border-radius:8px;\n            color:#c9e8ff;\n            margin-bottom:12px;\n        \">\n        <strong>■ MACD — Trend Confidence</strong><br>\n        • Upward crossover → Strong buying zone<br>\n        • Downward crossover → Weakening trend<br>\n        • Great for breakout confirmation\n        </div>\n        \"\"\", unsafe_allow_html=True)\n\n    st.markdown(\"\"\"\n    <div style=\"\n        background:rgba(0,180,216,0.05);\n        border-left:4px solid #00b4d8;\n        padding:14px 18px;\n        border-radius:8px;\n        margin-top:6px;\n        color:#aee6ff;\n    \">\n    <strong>■ Why this matters?</strong><br>\n    You learn trend-following strategies and understand <strong>market psychology</strong> —\n    helping you decide <strong>when to enter</strong> and <strong>when to exit</strong> smartly.\n    </div>\n    \"\"\", unsafe_allow_html=True)\n\n    view_mode = st.radio(\n        \"Indicator Display Mode\",\n        [\"Overlay on Chart\", \"Separate Panels\"],\n        horizontal=True,\n    )\n\n    ma_options = {\n        \"20 SMA\": \"SMA_20\",\n        \"50 SMA\": \"SMA_50\",\n        \"200 SMA\": \"SMA_200\",\n        \"20 EMA\": \"EMA_20\",\n        \"50 EMA\": \"EMA_50\",\n    }\n\n    overlay_selected = st.multiselect(\n        \"Select Moving Averages\",\n        list(ma_options.keys()),\n        default=[\"20 SMA\", \"50 SMA\"],\n    )\n\n    osc_options = [\"RSI (14)\", \"MACD\"]\n    osc_selected = st.multiselect(\n        \"Oscillators (separate charts)\",\n        osc_options,\n        default=[\"RSI (14)\"],\n    )\n\n    for ticker in selected_companies:\n        df = fetch_prices(ticker, start_date, end_date)\n        if df is None or df.empty:\n            continue\n\n        df = add_technical_indicators(df)\n        col_close = get_close_price_column(df)\n        last_valid_close = df[col_close].dropna().iloc[-1] if df[col_close].notna().any() else None\n\n        st.markdown(f\"### ■ {ticker}\")\n        st.metric(\n            \"Latest Close Price\",\n            f\"■ {last_valid_close:.2f}\" if last_valid_close is not None else \"Unavailable\",\n        )\n\n        if view_mode == \"Overlay on Chart\":\n            fig = go.Figure()\n            fig.add_trace(go.Scatter(x=df.trade_date, y=df[col_close],\n                                     name=\"Close Price\", mode=\"lines\"))\n            for label in overlay_selected:\n                col = ma_options[label]\n                if col in df.columns:\n                    fig.add_trace(go.Scatter(x=df.trade_date, y=df[col],\n                                             name=label, mode=\"lines\"))\n            fig.update_layout(\n                title=f\"{ticker} — Price + Indicators\",\n                height=500,\n                margin=dict(t=80, b=10),\n                legend=dict(orientation=\"h\", yanchor=\"bottom\", y=1.10,\n                            xanchor=\"center\", x=0.5),\n                xaxis_title=\"Date\",\n                yaxis_title=\"Price (■)\",\n            )\n            st.plotly_chart(fig, use_container_width=True)\n            download_csv(df, f\"{ticker}_overlay_data\")\n\n        else:\n            st.markdown(\"#### ■ Close Price\")\n            st.line_chart(df.set_index(\"trade_date\")[col_close], height=350)\n            for label in overlay_selected:\n                col = ma_options[label]\n                if col in df.columns:\n                    st.markdown(f\"#### ■ {label}\")\n                    st.line_chart(df.set_index(\"trade_date\")[[col_close, col]], height=300)\n\n        if \"RSI (14)\" in osc_selected:\n            st.markdown(\"#### ■ RSI (14)\")\n            st.line_chart(df.set_index(\"trade_date\")[\"RSI_14\"], height=250)\n\n        if \"MACD\" in osc_selected:\n            st.markdown(\"#### ■ MACD Indicator\")\n            st.line_chart(df.set_index(\"trade_date\")[\"MACD\"], height=250)\n\n        with st.expander(\"■ View & Download Data\"):\n            st.dataframe(df, use_container_width=True)\n            download_csv(df, f\"{ticker}_indicators\")\n\n\n# ================ TAB 2 — Abrupt Changes ==============\nwith tab2:\n    st.subheader(\"Sudden Price Jumps & Falls ■\")\n    st.markdown(f\"\"\"\n    <div style=\"\n        background:rgba(255,136,0,0.08);\n        border:1px solid rgba(255,180,50,0.4);\n        border-radius:10px;\n        padding:12px 18px;\n        text-align:right;\n        font-size:14.5px;\n        line-height:1.45;\n        color:{text_color};\n        margin:0px 0px 14px 0px;\n    \">\n        ■ Sudden price swings occur due to <strong>breaking news</strong>,\n        <strong>big-money trades</strong>, or <strong>volume spikes</strong>.<br>\n        ■ Upward jumps with strength → potential <strong>early breakout</strong>.<br>\n        ■ Sharp drops → fear selling & <strong>trend reversal</strong> risk.\n    </div>\n\n    <div style=\"\n        background:rgba(255,136,0,0.07);\n        border-left:4px solid #ff8800;\n        border-radius:8px;\n        padding:10px 16px;\n        text-align:left;\n        font-size:14.5px;\n        line-height:1.45;\n        color:{text_color};\n    \">\n        <strong>■ Why this matters?</strong><br>\n        Spot <strong>breakout chances</strong> early or protect yourself before\n        <strong>fast crashes</strong> cause damage ■■.\n    </div>\n    \"\"\", unsafe_allow_html=True)\n\n    threshold_pct = st.slider(\"Threshold (%)\", 3, 20, 7) / 100\n\n    for ticker in selected_companies:\n        df = fetch_prices(ticker, start_date, end_date)\n        if df is None or df.empty:\n            continue\n        col_close = get_close_price_column(df)\n        abrupt = detect_abrupt_changes(df, threshold_pct)\n\n        st.markdown(f\"### {ticker} — Abrupt Moves\")\n        if abrupt.empty:\n            st.info(\"No major movements detected.\")\n            continue\n\n        fig = px.bar(\n            abrupt,\n            x=\"trade_date\",\n            y=\"pct_change\",\n            color=\"pct_change\",\n            color_continuous_scale=\"RdYlGn\",\n            title=f\"Abrupt % Movements — {ticker}\",\n        )\n        fig.update_layout(xaxis_title=\"Date\", yaxis_title=\"Daily % Change\",\n                          coloraxis_colorbar_title=\"% Change\")\n        st.plotly_chart(fig, use_container_width=True)\n        st.dataframe(abrupt, use_container_width=True)\n\n\n# ================ TAB 3 — Risk View ==============\nwith tab3:\n    st.subheader(\"Volatility & Risk Index\")\n    st.markdown(\"\"\"\n    <div style=\"\n        background:linear-gradient(135deg, rgba(255,40,40,0.10), rgba(180,0,0,0.08));\n        padding:16px 18px;\n        border-radius:10px;\n        border-left:4px solid #ff4d4d;\n        margin-bottom:10px;\n        color:#ffdddd;\n    \">\n    <strong>■ Volatility means uncertainty:</strong><br>\n    • Big price swings → confusion / hype ■■■<br>\n    • High risk during panic buying/selling ■<br>\n    • Sudden drops → instability warning ■<br>\n    </div>\n\n    <div style=\"\n        background:rgba(255,255,255,0.04);\n        padding:14px 18px;\n        border-radius:10px;\n        border-left:4px solid #ff9f43;\n        margin-bottom:10px;\n        color:#ffe9c7;\n    \">\n    <strong>■ How we measure Risk:</strong><br>\n    • Volatility Index (fear indicator) ■<br>\n    • Price variance → instability levels ■<br>\n    • Shorter time periods → more sensitivity ■■<br>\n    </div>\n\n    <div style=\"\n        background:rgba(255,255,255,0.03);\n        padding:14px 18px;\n        border-radius:10px;\n        border-left:4px solid #ffd166;\n        color:#fff7d6;\n    \">\n    <strong>■ Why this matters?</strong><br>\n    Volatility helps you judge <strong>when to reduce position size</strong> and\n    <strong>protect capital</strong> when the market heats up ■■.\n    </div>\n    \"\"\", unsafe_allow_html=True)\n\n    window = st.slider(\"Volatility Window\", 5, 55, 20)\n    for ticker in selected_companies:\n        df = fetch_prices(ticker)\n        if df is None or df.empty:\n            continue\n        vr = volatility_and_risk(df, window)\n        st.markdown(f\"### {ticker} — Volatility vs Risk\")\n        st.line_chart(vr.set_index(\"trade_date\")[[\"volatility\", \"risk\"]],\n                      use_container_width=True)\n\n\n# ============== TAB 4 — Comparison View ==============\nwith tab4:\n    st.subheader(\"Compare Price Trends & Correlation\")\n    st.markdown(\"\"\"\n    <div style=\"margin-top:10px;\">\n    <div style=\"\n        background:linear-gradient(145deg, #005bea33, #00c6fb22);\n        padding:16px 20px;\n        border-radius:10px;\n        color:#e8f8ff;\n        margin-bottom:12px;\n        border-left:4px solid #00c6fb;\n    \">\n    <strong>■ Compare Price Strength</strong><br>\n    • Normalize price → fair comp. ■<br>\n    • Spot consistent outperformers ■<br>\n    • Check who beats the market over time ■■<br>\n    </div>\n\n    <div style=\"\n        background:linear-gradient(145deg, #b721ff33, #21d4fd22);\n        padding:16px 20px;\n        border-radius:10px;\n        color:#f1e6ff;\n        margin-bottom:14px;\n        border-left:4px solid #9b59ff;\n    \">\n    <strong>■ Correlation Outlook</strong><br>\n    • Same direction = paired performance ■<br>\n    • Opposite direction = hedge opportunities ■<br>\n    • Build diversified portfolios with confidence ■■<br>\n    </div>\n\n    <div style=\"\n        background:rgba(0,200,255,0.06);\n        padding:12px 16px;\n        border-radius:10px;\n        border-left:4px solid #00c6fb;\n        color:#c7f4ff;\n    \">\n    <strong>■ Why this matters?</strong><br>\n    Avoid <strong>putting all eggs in one basket</strong> → diversify smartly to reduce risk ■■■.\n    </div>\n    </div>\n    \"\"\", unsafe_allow_html=True)\n\n    if len(selected_companies) >= 2:\n        merged = compare_companies(selected_companies, start_date, end_date)\n        st.markdown(\"### ■ Normalized Price Comparison\")\n\n        fig = go.Figure()\n        for ticker in selected_companies:\n            if ticker in merged.columns:\n                fig.add_trace(go.Scatter(x=merged.index, y=merged[ticker],\n                                         mode=\"lines\", name=ticker))\n        fig.update_layout(\n            xaxis_title=\"Date\",\n            yaxis_title=\"Normalized Price\",\n            title=\"Price Comparison Across Selected Stocks\",\n            height=550,\n            margin=dict(l=0, r=0, t=40, b=80),\n            legend=dict(orientation=\"h\", yanchor=\"top\", y=-0.15,\n                        xanchor=\"center\", x=0.5, title=None),\n        )\n        st.plotly_chart(fig, use_container_width=True)\n        download_csv(merged.reset_index(), \"normalized_price_comparison\")\n\n        st.markdown(\"### ■ Correlation Matrix\")\n        corr = correlation_analysis(selected_companies)\n        st.dataframe(corr.style.background_gradient(cmap=\"coolwarm\"))\n        plot_correlation(corr)\n\n\n# ============== TAB 5 — Smart Insights ==============\nwith tab5:\n    st.subheader(\"■ Smart Insights, Opportunities & Forecast\")\n    st.markdo
```

```
wn(\"### ■ Understand AI-Driven Market Signals\")\n\n    col1, col2 = st.columns(2)\n    with col1:\n        st.markdown(\"\"\"\n        <div style=\"\n            background:rgba(0,255,200,0.08);\n            border-left:4px solid #00ffc8;\n            padding:12px 16px;\n            border-radius:8px;\n            color:#b9fff4;\n            margin-bottom:12px;\n        \">\n            <strong>■ AI Trend Confidence</strong><br>\n            • Tags a stock as Buy / Hold / Risky ■<br>\n            • Reduces emotional trading mistakes\n        </div>\n        \"\"\", unsafe_allow_html=True)\n\n    with col2:\n        st.markdown(\"\"\"\n        <div style=\"\n            background:rgba(0,255,200,0.08);\n            border-left:4px solid #00ffc8;\n            padding:12px 16px;\n            border-radius:8px;\n            color:#b9fff4;\n            margin-bottom:12px;\n        \">\n            <strong>■ Future Price Tendency</strong><br>\n            • Shows short-term directional pressure<br>\n            • Improves timing for smarter entries\n        </div>\n        \"\"\", unsafe_allow_html=True)\n\n    col3, col4 = st.columns(2)\n    with col3:\n        st.markdown(\"\"\"\n        <div style=\"\n            background:rgba(0,255,200,0.08);\n            border-left:4px solid #00ffc8;\n            padding:12px 16px;\n            border-radius:8px;\n            color:#b9fff4;\n            margin-bottom:12px;\n        \">\n            <strong>■ Buy & Sell Zones</strong><br>\n            • Shows where reversals are likely<br>\n            • Avoid buying tops and selling bottoms\n        </div>\n        \"\"\", unsafe_allow_html=True)\n\n    with col4:\n        st.markdown(\"\"\"\n        <div style=\"\n            background:rgba(0,255,200,0.08);\n            border-left:4px solid #00ffc8;\n            padding:12px 16px;\n            border-radius:8px;\n            color:#b9fff4;\n            margin-bottom:12px;\n        \">\n            <strong>■ Share Affordability</strong><br>\n            • Teaches sizing and discipline<br>\n            • Plan smarter investments\n        </div>\n        \"\"\", unsafe_allow_html=True)\n\n    st.markdown(\"\"\"\n    <div style=\"\n        background:rgba(0,255,200,0.05);\n        border-left:4px solid #00ffc8;\n        padding:14px 18px;\n        border-radius:8px;\n        margin-top:6px;\n        color:#a6fff5;\n    \">\n        <strong>■ Why this matters?</strong><br>\n        Builds <strong>high-confidence decisions</strong> and prevents\n        <strong>FOMO or panic selling</strong> — clarity on\n        <strong>when to enter</strong> & <strong>exit</strong> the market like a smart investor.\n    </div>\n    \"\"\", unsafe_allow_html=True)\n    st.caption(\"Educational purpose only — Not for financial decisions ■\")\n\n    budget_vals, budget_labels = build_budget_options()\n    budget = budget_vals[4]\n    budget_label = budget_labels[4]\n    horizon = \"Short Term\"\n\n    colA, colB, colC = st.columns(3)\n    with colA:\n        budget_label = st.selectbox(\"Budget\", budget_labels, index=4)\n        budget = budget_vals[budget_labels.index(budget_label)]\n    with colB:\n        horizon = st.radio(\"Type\", [\"Short Term\", \"Long Term\"], horizontal=True)\n    with colC:\n        forecast_window = 15 if horizon == \"Short Term\" else 60\n        st.metric(\"Forecast Window\", f\"{forecast_window} days\")\n\n    st.caption(\"Based on trends — Not financial advice. Educational use only.\")\n\n    for ticker in selected_companies:\n        df = fetch_prices(ticker)\n        if df is None or df.empty:\n            continue\n\n        df = compute_sma(df)\n        col_close = get_close_price_column(df)\n\n        conf, label, pct, vol = analyze_trend_confidence(df, col_close, horizon)\n        valid_prices = (\n            pd.to_numeric(df[col_close], errors=\"coerce\")\n            .replace([np.inf, -np.inf], np.nan)\n            .dropna()\n        )\n        if valid_prices.empty or valid_prices.iloc[-1] <= 0:\n            st.warning(f\"{ticker}: no valid latest closing price is available.\")\n            continue\n\n        latest = float(valid_prices.iloc[-1])\n        shares = max(1, int(budget / latest))\n\n        st.markdown(f\"---\n### {ticker}\")\n        col1, col2, col3, col4 = st.columns(4)\n        col1.metric(\"Signal\", label)\n        col2.metric(\"Confidence\", f\"{conf:.1f}%\")\n        col3.metric(\"Trend %\", f\"{pct:+.2f}%\")\n        col4.metric(\"Volatility\", f\"{vol:.2f}%\")\n\n        st.caption(f\"With {budget_label}, Approx shares possible: **{shares}**\")\n\n        buy_future, sell_future = project_future(df, col_close, horizon)\n\n        fig = go.Figure()\n        fig.add_trace(go.Scatter(x=df[\"trade_date\"], y=df[col_close],\n                                 mode=\"lines\", name=\"Close Price\"))\n        fig.add_trace(go.Scatter(x=df[\"trade_date\"], y=df[\"SMA\"],\n                                 mode=\"lines\", name=\"SMA\"))\n\n        if buy_future is not None and not buy_future.empty:\n            fig.add_trace(go.Scatter(x=buy_future[\"trade_date\"],\n                                     y=buy_future[col_close],\n                                     mode=\"markers\", name=\"Future Buy\",\n                                     marker_color=\"green\"))\n        if sell_future is not None and not sell_future.empty:\n            fig.add_trace(go.Scatter(x=sell_future[\"trade_date\"],\n                                     y=sell_future[col_close],\n                                     mode=\"markers\", name=\"Future Sell\",\n                                     marker_color=\"red\"))\n\n        fig.update_layout(title=f\"{ticker} — Forecasted Buy/Sell Zones\")\n        st.plotly_chart(fig, use_container_width=True)\n\n        with st.expander(\"■ Forecasted Opportunities\"):\n            buy_col, sell_col = st.columns(2)\n\n            buy_col.markdown(\"#### ■ Future Buy Opportunities\")\n            if buy_future is None or buy_future.empty:\n                buy_col.info(\"No buy signals ahead ■\")\n            else:\n                buy_col.dataframe(buy_future[[\"trade_date\", col_close]],\n                                  use_container_width=True)\n\n            sell_col.markdown(\"#### ■ Future Sell Opportunities\")\n            if sell_future is None or sell_future.empty:\n                sell_col.info(\"No sell signals detected ■\")\n            else:\n                sell_col.dataframe(sell_future[[\"trade_date\", col_close]],\n                                   use_container_width=True)\n\n\n# ============================================================\n# TAB 6 — DWM: DATA EXPLORATION & PREPROCESSING\n# ============================================================\nwith tab6:\n    st.subheader(\"■ Data Exploration & Preprocessing (DWM Module 3)\")\n    st.caption(\"Uses the selected stock's stored OHLCV data. This is an educational DWM workflow, not just a stock indicator view.\")\n    eda_ticker = st.selectbox(\"Ticker for EDA\", selected_companies, key=\"dwm_eda_ticker\")\n    raw_eda = fetch_prices(eda_ticker, start_date, end_date)\n    if raw_eda is None or raw_eda.empty:\n        st.warning(\"No rows are available for this ticker/date range.\")\n    else:\n        try:\n            raw_summary = summarize_dwm_dataset(raw_eda)\n            cleaned_eda = prepare_dwm_dataset(raw_eda)\n            clean_summary = summarize_dwm_dataset(cleaned_eda)\n            c1, c2, c3 = st.columns(3)\n            c1.metric(\"Raw Rows\", f\"{len(raw_eda):,}\")\n            c2.metric(\"Cleaned Rows\", f\"{len(cleaned_eda):,}\")\n            c3.metric(\"Derived Attributes\", f\"{max(0, cleaned_eda.shape[1] - raw_eda.shape[1])}\")\n            st.markdown(\"#### Dataset quality before preprocessing\")\n            st.dataframe(raw_summary[\"overview\"], use_container_width=True, hide_index=True)\n            st.markdown(\"#### Missing values and attribute types (cleaned dataset)\")\n            st.dataframe(clean_summary[\"missing\"], use_container_width=True, hide_index=True)\n            st.markdown(\"#### Statistical description\")\n            st.dataframe(clean_summary[\"statistics\"], use_container_width=True, hide_index=True)\n            numeric_cols = cleaned_eda.select_dtypes(include=np.number).columns.tolist()\n            default_col = \"close\" if \"close\" in numeric_cols else numeric_cols[0]\n            chart_col = st.selectbox(\"Numeric attribute to visualize\", numeric_cols, index=numeric_cols.index(default_col), key=\"dwm_eda_chart_col\")\n            chart_kind = st.radio(\"Visualization\", [\"Line chart\", \"Histogram\", \"Box plot\"], horizontal=True, key=\"dwm_eda_chart_kind\")\n            if chart_kind == \"Line chart\" and \"trade_date\" in cleaned_eda:\n                fig = px.line(cleaned_eda, x=\"trade_date\", y=chart_col, title=f\"{chart_col} over time\")\n            elif chart_kind == \"Histogram\":\n                fig = px.histogram(cleaned_eda, x=chart_col, title=f\"Distribution of {chart_col}\")\n            else:\n                fig = px.box(cleaned_eda, y=chart_col, points=\"outliers\", title=f\"Box plot of {chart_col}\")\n            st.plotly_chart(fig, use_container_width=True)\n            with st.expander(\"View preprocessed rows\"):\n                st.dataframe(cleaned_eda, use_container_width=True)\n            download_csv(cleaned_eda, f\"{eda_ticker}_dwm_preprocessed\")\n            st.info(\"Cleaning applied: date sorting, duplicate-date removal, numeric conversion, invalid close removal, non-finite handling, and median imputation for numeric features. Derived columns include returns, SMA/EMA, volatility, and next-day targets.\")\n        except Exception as exc:\n            st.error(f\"Error during data exploration and preprocessing: {exc}\")\n\n\n# ============================================================\n# TAB 7 — DWM: CLASSIFICATION & REGRESSION\n# ============================================================\nwith tab7:\n    st.subheader(\"■ Classification & Regression (DWM Module 4)\")\n    st.markdown(\"Predict **next-day direction** with Decision Tree and Naive Bayes, and estimate **next-day closing price** with multiple linear regression. Evaluation uses a chronological train/test split.\")\n    ml_ticker = st.selectbox(\"Ticker for supervised mining\", selected_companies, key=\"dwm_ml_ticker\")\n    ml_raw = fetch_prices(ml_ticker, start_date, end_date)\n    if ml_raw is None or ml_raw.empty:\n        st.warning(\"No data is available for supervised learning.\")\n    else:\n        try:\n            with st.spinner(\"Training DWM demonstration models...\"):\n                ml_result = run_dwm_supervised_analysis(ml_raw)\n            st.markdown(\"#### Regression — next-day closing price\")\n            st.dataframe(ml_result[\"regression_metrics\"], use_container_width=True, hide_index=True)\n            pred_df = ml_result[\"regression_predictions\"]\n            fig = go.Figure()\n            fig.add_trace(go.Scatter(x=pred_df[\"trade_date\"], y=pred_df[\"Actual Next Close\"], mode=\"lines\", name=\"Actual\"))\n            fig.add_trace(go.Scatter(x=pred_df[\"trade_date\"], y=pred_df[\"Predicted Next Close\"], mode=\"lines\", name=\"Predicted\"))\n            fig.update_layout(title=\"Actual vs Predicted Next Close\", xaxis_title=\"Date\", yaxis_title=\"Price\")\n            st.plotly_chart(fig, use_container_width=True)\n            st.markdown(\"#### Classification — next-day direction\")\n            st.dataframe(ml_result[\"classification_metrics\"], use_container_width=True, hide_index=True)\n            st.caption(f\"Chronological split: {ml_result['train_rows']:,} training rows / {ml_result['test_rows']:,} testing rows. Features: {', '.join(ml_result['feature_columns'])}.\")\n            download_csv(ml_result[\"classification_metrics\"], f\"{ml_ticker}_classification_metrics\")\n            download_csv(pred_df, f\"{ml_ticker}_regression_predictions\")\n            st.warning(\"These are historical hold-out metrics, not a promise of future accuracy. The Decision Tree uses entropy (ID3-style); it is not a complete from-scratch implementation of textbook ID3.\")\n        except Exception as exc:\n            st.error(f\"Could not run supervised analysis: {exc}\")\n\n\n# ============================================================\n# TAB 8 — DWM: CLUSTERING\n# ============================================================\nwith tab8:\n    st.subheader(\"■ Stock Profile Clustering (DWM Module 5)\")\n    st.write(\"Group selected companies using return, volatility, volume, and price-range features. Features are standardized before clustering.\")\n    cluster_method = st.selectbox(\"Clustering method\", [\"K-Means\", \"Hierarchical (Agglomerative)\"], key=\"dwm_cluster_method\")\n    if len(selected_companies) < 2:\n        cluster_count = 2\n        st.info(\"Select at least two companies in the sidebar to compare
```

clusters.\")\n   else:\n   max_clusters = min(6, len(selected_companies))\n   if max_clusters == 2:\n   cluster_count = 2\n   st.caption(\"Two companies selected; clustering will use two clusters.\")\n   else:\n   cluster_count = st.slider(\"Number of clusters\", 2, max_clusters, min(3, max_clusters), key=\"dwm_cluster_count\")\n   history_map = {ticker: fetch_prices(ticker, start_date, end_date) for ticker in selected_companies}\n   try:\n   profiles = build_stock_profiles(history_map)\n   clustered = cluster_stock_profiles(profiles, cluster_count, cluster_method)\n   st.dataframe(clustered, use_container_width=True, hide_index=True)\n   if not clustered.empty:\n   fig = px.scatter(clustered, x=\"Average Daily Return %\", y=\"Return Volatility %\", color=clustered[\"Cluster\"].astype(str), hover_name=\"Ticker\", size=\"Price Range %\", title=f\"{cluster_method} — Return vs Volatility\")\n   fig.update_layout(legend_title=\"Cluster\")\n   st.plotly_chart(fig, use_container_width=True)\n   st.markdown(\"#### Cluster interpretation\")\n   interpretation = clustered.groupby(\"Cluster\").agg(Companies=(\"Ticker\", lambda s: \", \".join(s)), Mean_Return_Pct=(\"Average Daily Return %\", \"mean\"), Mean_Volatility_Pct=(\"Return Volatility %\", \"mean\"), Mean_Total_Return_Pct=(\"Total Return %\", \"mean\")).reset_index().round(3)\n   st.dataframe(interpretation, use_container_width=True, hide_index=True)\n   download_csv(clustered, \"dwm_stock_clusters\")\n   except Exception as exc:\n   st.error(f\"Clustering could not run: {exc}\")\n\n\n# ==\n# TAB 9 — DWM: FREQUENT PATTERNS & ASSOCIATION RULES\n# ==\nwith tab9:\n   st.subheader(\"■ Frequent Pattern Mining — Apriori (DWM Module 6)\")\n   st.write(\"Each trading date is treated as a transaction containing selected ticker states (UP/DOWN/FLAT, HIGH/LOW_VOLUME, HIGH/LOW_MOVE). Rules describe historical co-occurrence, not causation.\")\n   support_pct = st.slider(\"Minimum support (%)\", 1, 60, 10, key=\"dwm_apriori_support\")\n   confidence_pct = st.slider(\"Minimum confidence (%)\", 10, 100, 50, key=\"dwm_apriori_confidence\")\n   max_items = st.select_slider(\"Maximum itemset size\", options=[2, 3], value=3, key=\"dwm_apriori_size\")\n   try:\n   pattern_histories = {ticker: fetch_prices(ticker, start_date, end_date) for ticker in selected_companies}\n   transactions = build_market_transactions(pattern_histories)\n   st.metric(\"Daily transactions\", f\"{len(transactions):,}\")\n   if not transactions:\n   st.info(\"No transactions could be built for the selected date range.\")\n   else:\n   itemsets, rules = mine_association_rules(transactions, support_pct / 100, confidence_pct / 100, max_items)\n   if itemsets.empty:\n   st.info(\"No frequent itemsets meet the selected support threshold. Lower minimum support and retry.\")\n   else:\n   st.markdown(\"#### Frequent itemsets\")\n   st.dataframe(itemsets, use_container_width=True, hide_index=True)\n   download_csv(itemsets, \"dwm_frequent_itemsets\")\n   st.markdown(\"#### Association rules\")\n   if rules.empty:\n   st.info(\"No rules meet both thresholds. Lower support/confidence or select more tickers.\")\n   else:\n   display_rules = rules.copy()\n   for col in [\"Support\", \"Confidence\"]:\n   display_rules[col] = (display_rules[col] * 100).round(2).astype(str) + \"%\"\n   display_rules[\"Lift\"] = display_rules[\"Lift\"].round(3)\n   st.dataframe(display_rules, use_container_width=True, hide_index=True)\n   download_csv(rules, \"dwm_association_rules\")\n   st.caption(\"Support = fraction of trading dates containing both sides. Confidence = frequency of the consequent when the antecedent occurs. Lift > 1 indicates a positive association in this sample.\")\n   except Exception as exc:\n   st.error(f\"Error during association rule mining: {exc}\")\n\n\n# ==\n# TAB 10 — DWM: OLAP-STYLE ANALYSIS\n# ==\nwith tab10:\n   st.subheader(\"■ OLAP Cube — Slice, Dice, Roll-up & Drill-down\")\n   st.write(\"Explore stored stock facts by ticker and time. Yearly aggregation is a roll-up; quarterly/monthly views drill down; ticker and direction filters perform slice/dice.\")\n   try:\n   olap_histories = {ticker: fetch_prices(ticker, start_date, end_date) for ticker in selected_companies}\n   fact_table = build_olap_fact_table(olap_histories)\n   if fact_table.empty:\n   st.warning(\"No fact rows are available for OLAP aggregation.\")\n   else:\n   col_a, col_b, col_c = st.columns(3)\n   with col_a:\n   olap_level = st.selectbox(\"Time granularity\", [\"Yearly\", \"Quarterly\", \"Monthly\"], index=2, key=\"dwm_olap_level\")\n   with col_b:\n   olap_tickers = st.multiselect(\"Slice by ticker\", selected_companies, default=selected_companies, key=\"dwm_olap_tickers\")\n   with col_c:\n   olap_direction = st.selectbox(\"Dice by direction\", [\"All\", \"Up\", \"Down\", \"Flat\"], key=\"dwm_olap_direction\")\n   olap_result = olap_aggregate(fact_table, olap_level, olap_tickers, olap_direction)\n   if olap_result.empty:\n   st.info(\"No rows match the current OLAP filters.\")\n   else:\n   st.dataframe(olap_result, use_container_width=True, hide_index=True)\n   st.markdown(\"#### Average close by period\")\n   period_cols = [c for c in [\"Year\", \"Quarter\", \"Month\"] if c in olap_result.columns]\n   olap_result[\"Period\"] = olap_result[period_cols].astype(str).agg(\"-\".join, axis=1)\n   fig = px.bar(olap_result, x=\"Period\", y=\"Average_Close\", color=\"Ticker\", barmode=\"group\", title=\"Average close by OLAP period\")\n   st.plotly_chart(fig, use_container_width=True)\n   download_csv(olap_result, \"dwm_olap_aggregate\")\n   with st.expander(\"View underlying fact table\"):\n   st.dataframe(fact_table, use_container_width=True, hide_index=True)\n   download_csv(fact_table, \"dwm_stock_fact_table\")\n   except Exception as exc:\n   st.error(f\"Error during OLAP cube aggregation: {exc}\")\n\",

  "patch": "",

  "confidence": 0.98,

  "provider": "gemini",

  "model": "gemini-3.5-flash-lite"

}

6. Verification

{

  "verified": true,

  "method": "static",

  "reason": "Target file passed available local syntax verification.",

  "developer_reason": "changed app.py to include the changes done in calculations.py so that the frontend doesn't face any more errors."

}

7. Security Before

{}

8. Security After

{}

9. Risk Before

{}

10. Risk After

{}

11. Risk Comparison

{}

12. Security Gate

{
  "allowed": true,
  "mode": "pending-security-pipeline",
  "reason": "Static verification passed. A synthetic security risk profile was not created by this workflow."
}

13. Git Audit

{
  "original_branch": "main",
  "candidate_branch": "sentinelai/security-remediation-20261013-113931",
  "candidate_path": "/tmp/sentinelai-remediation-ufzrmfi_/Stock-Insights",
  "ai_candidate_applied": false,
  "remediation_commit_sha": "0b880ad38a9b2dcd5aebed84ed2dff530f5e2adc"
}

14. GitHub Audit

{}

15. Tests

{
  "candidate_changed": true,
  "static_verification": {
    "verified": true,
    "method": "static",
    "reason": "Target file passed available local syntax verification.",
    "developer_reason": "changed app.py to include the changes done in calculations.py so that the frontend doesn't face any more errors."
  }
}